

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to **Ankit Sir** for his valuable guidance, continuous support, and encouragement throughout the completion of our project, “**Mobile Price Range Classification.**” His insights and suggestions helped us understand the concepts of machine learning models and apply them effectively. We are also thankful to our institution for providing us with the necessary resources and a collaborative environment to work on this project.

We appreciate the dedication and efforts of all our group members, whose contributions made this project successful:

- **Palak Machhiwal** – Report Writing & Documentation
- **Yash Pratap Singh Thakur** – Coder / ML Model Implementation
- **Bhoomi Agarwal** – Presentation Designing
- **Chandan Karan** – Data Collection & Data Management
- **Hrishab Jagtap** – Model Evaluation & Testing

Each member worked with responsibility and teamwork, ensuring the timely and accurate completion of the project.

Finally, we would like to thank everyone who directly or indirectly supported us during this project. Their help and encouragement motivated us to give our best.

Table of Contents

S.N O	CONTENT
1.	Introduction Of Project
2.	Problem Statement
3.	Dataset Description
4.	Data Collection
5.	Data Preprocessing Data Checking Data Cleaning:- Verify Feature Unit:- Unit Conversion From Mb To Gb:- Exploratory Data Analysis (Eda) Feature Engineering:- Encoding The Target Variable Dataset Splitting (Train-Test Split) Standardization:-
6.	Train All Five Model:-
7.	Hyper Tuning :-
8.	Cross-Validation Comparison:-

9.	Finale Test Evaluation:-
10.	Confusion Matrix:-
11.	Classification Report

INTRODUCTION OF PROJECT

- WHAT IS MOBILE PRICE CLASSIFICATION?

Mobile Price Classification is a machine learning task in which mobile phones are categorized into different price ranges such as low, medium, or high based on their technical specifications. By analyzing features like RAM, storage, processor score, and battery power, a model can predict which price category a mobile belongs to. This helps in understanding the market value of the device using data-driven methods.

- WHY IS IT NEEDED?

Mobile Price Classification is needed because it helps identify the pricing segment of a mobile phone without manually checking every specification.

- **For consumers**, it provides clarity on whether a mobile is budget-friendly, mid-range, or premium.
- **For manufacturers and sellers**, it supports pricing strategy, product positioning, and competitive analysis.
- **For analysts**, it helps study trends and understand how different features impact price.
Overall, it automates and simplifies the process of determining a mobile's price category.

- WHAT FEATURES ARE USED?

Main features used in the project:

- **RAM:** Measures how well the phone can multitask.
- **Storage:** Determines how much data/apps the phone can hold.

- **Processor Score:** Indicates the phone's processing speed and power (clock speed + number of cores).
- **Battery Power:** Represents battery capacity and expected usage time.
- **Price Range:** The target variable representing low, mid, or high price category.

In summary, mobile price classification helps us understand how key technical features determine the overall price category of a mobile phone, making the prediction process accurate and efficient.

Problem Statement

The objective of this project is to build a **machine learning-based classification system** that can accurately predict the **price range of a mobile phone** as **Low, Medium, or High**.

The classification is performed using key technical specifications of mobile phones, including **RAM, internal storage, and processor performance score**. These features play a major role in determining the cost and market category of a mobile device.

To achieve this objective, the project implements and compares the following **five machine learning models**:

- Linear Regression / Classification
- Logistic Regression/Classification
- Decision Tree
- Random Forest
- K-Nearest Neighbors (KNN)

Each model is trained and evaluated to determine its effectiveness in classifying mobile phones into the correct price range. The final goal is to **identify the most accurate and reliable model** based on performance metrics such as accuracy, confusion matrix, and classification report.

Dataset Description

- **Dataset Description**

The dataset contains specifications of various mobile phones along with their price range category. Each row represents one mobile device, and each column provides a different feature such as RAM, storage, processor score, battery power, etc. The dataset also includes a target column called **price_range**, which classifies mobiles into Low, Medium, or High price categories. This dataset is suitable for machine learning tasks related to classification.

- **Source**

The dataset is taken from publicly available mobile specification datasets typically used for mobile price prediction and machine learning research.

- **Features**

Main Features Used in the Project:

- **RAM**
- **Storage (internal memory)**
- **Processor Score (clock_speed + number of cores)**
- **Battery Power**

- **Price Range (target variable)**

Other Features (Only Names):

Bluetooth (blue), Dual SIM (dual_sim), Front Camera (fc), 3G (three_g), 4G (four_g), Rear Camera (pc), Screen Height (sc_h), Screen Width (sc_w), Mobile Weight (mobile_wt), Touch Screen (touch_screen), WiFi (wifi), Pixel Height (px_height), Pixel Width (px_width), Mobile Depth (m_dep), Talk Time (talk_time)

Why We Selected These Features

The main features—RAM, storage, processor score, and battery power—were selected because they are the most important factors that directly influence a mobile phone's performance and price. RAM and processor determine the speed of the device, storage affects usability, and battery power impacts user experience. These features are highly correlated with mobile pricing and allow machine learning models to classify price ranges accurately. The other features were included in the dataset but were not the primary focus of our project.

DATA COLLECTION

SOURCE OF DATA:

- GitHub

DATASET SIZE:

- ROWS: 2000
- COLUMNS: 21

COLUMN NAME:

- Battery_power
- blue
- clock_speed
- dual_sim
- fc
- four_g
- int_memory
- m_dep
- mobile_wt
- n_cores
- pc
- px_height
- px_width
- ram

- sc_h
- sc_w
- talk_time
- three_g
- touch_screen
- Wifi
- price_range (target)

Q.1 WHY 2000 ROWS?

- *Dataset is public benchmark for mobile price classification.*
- *2000 samples are enough to:*
 1. *Train ML models reliably*
 2. *Avoid overfitting*
 3. *Run cross-validation properly*
- *Balanced classes (low/mid/high/veryhigh) → fair learning.*

Q.2 WHY 21 COLUMNS?

- *20 input features describe a phone completely:*
 1. *Hardware (RAM, battery, cores, clock)*
 2. *Display (px_height, px_width, sc_h, sc_w)*
 3. *Connectivity (3G, 4G, WiFi, Bluetooth)*
 4. *Physical (weight, depth)*
- *1 target column:*

price_range (low/mid/high/veryhigh)

Q.3 WHY NOT FEWER OR MORE?

- *Fewer features → information loss → poor accuracy.*
- *More features → noise, overfitting.*

- *This dataset already has optimal, engineered features.*

Q.4 WHY 4 CLASSES (LOW, MID, HIGH, VERY HIGH) INSTEAD OF 3 CLASSES (LOW/MID/HIGH)?

- **Better price resolution**
- *Mobile prices don't fall cleanly into just low–mid–high.*
- *“Very high” separates flagship phones from normal high-end phones.*
- **More realistic market segmentation**
- *Budget*
- *Mid-range*
- *Premium*
- *Flagship*
- **Richer learning for the model**
- *4 classes force the model to learn finer differences in specs.*
- *3 classes would merge high and flagship phones → information loss.*
- **Academic advantage**
- *Multiclass (4-class) classification looks stronger and more advanced*
- *Better for confusion matrix, CV, and evaluation discussion.*
- *Four price categories were used to capture finer distinctions between mobile phone price segments, especially to separate premium and flagship devices, which would be lost in a three-class setup.”*

Q.5 WHY WE USED 21 FEATURES WHEN PROBLEM STATEMENT MENTIONS ONLY RAM, PROCESSOR SCORE, AND STORAGE?

1) Problem statement is simplified

- *“RAM, processor score, storage” are primary indicators, not the only ones.*
- *Problem statements are abstracted versions of real-world pricing.*

2) Mobile price depends on multiple factors

- *Real pricing also depends on:*
- *Battery capacity, Screen resolution & size, Camera specs, Connectivity (4G/5G, WiFi)*
- *, Weight & build, CPU cores & clock speed*
- *Ignoring them would make the model unrealistic.*

3) ML models learn importance automatically

- *We don't force the model to use all features equally.*
- *Feature importance later shows:*
- *RAM & battery are dominant*
- *Others contribute less*
- *So irrelevant features are down-weighted automatically.*

4) Using fewer features can reduce accuracy

- *3-feature model = underfitting*
- *21-feature model = better representation*
- *We verified this using:*
- *Cross-validation*

DATA PREPROCESSING

- **DATA CHECKING**

Before training any machine learning model, the dataset is carefully checked to ensure data quality and reliability.

- **Checking for Missing Values:**
Missing values can confuse machine learning algorithms and lead to incorrect pattern learning, resulting in poor model accuracy. Therefore, the dataset is examined to detect any null or empty entries.
- **Checking for Duplicate Row:**
Duplicate records can bias the model by repeating the same information multiple times. This may cause overfitting and produce misleading performance results.
- **Purpose of Data Checking:**
The main purpose of checking missing values and duplicate rows is to ensure that the dataset is **clean, consistent, and unbiased** before applying preprocessing and model training.

```
# Load RAW dataset
df = pd.read_csv('/content/drive/MyDrive/2.0 Machine Lea

print("==== RAW DATA CHECK =====")
print("Initial dataset shape:", df.shape)

# Check missing values
missing_total = df.isnull().sum().sum()
print("Total missing values detected:", missing_total)

# Check duplicate rows
duplicate_total = df.duplicated().sum()
print("Total duplicate rows detected:", duplicate_total)

#total problematic rows
print("Total problematic rows:",duplicate_total+ missing
```

● **DATA CLEANING:-**

The purpose of data cleaning is to ensure that the dataset is **accurate, consistent, and suitable for machine learning modeling**. High-

quality data is essential for obtaining reliable and unbiased results from machine learning algorithms.

Handling Missing Values

Rows containing missing or null values were identified and removed from the dataset. Machine learning algorithms cannot operate reliably when data is incomplete, as missing values distort underlying patterns and negatively impact model accuracy. Removing such entries ensures that all models are trained on complete and meaningful data.

Removing Duplicate Records

Duplicate rows present in the dataset were removed. Duplicate data can introduce bias by over-representing certain samples, which may lead to overfitting and incorrect learning behavior. Eliminating duplicates ensures that each data instance contributes equally to the training process.

CODE :-

```
11 import pandas as pd

# 1. Remove missing values
df = df.dropna()

# 2. Remove duplicate rows
df = df.drop_duplicates()

# Show final cleaned shape
print("Cleaned dataset shape:", df.shape)

# Optional: preview
df.head()
```

OUTPUT:-

Cleaned dataset shape: (2000, 21)

	battery_power	blue	clock_speed	dual_sim	fc	four_g	int_memory	m_dep	mobile_wt	n_cores	...	px_height	px_width	ram	sc_h	sc_w	talk_time	three_g	touch_screen	wifi	price_range
0	842	0	2.2	0	1	0	7	0.6	188	2	...	20	756	2549	9	7	19	0	0	1	1
1	1021	1	0.5	1	0	1	53	0.7	136	3	...	906	1988	2631	17	3	7	1	1	0	2
2	563	1	0.5	1	2	1	41	0.9	145	5	...	1263	1716	2603	11	2	9	1	1	0	2
3	615	1	2.5	0	0	0	10	0.8	131	6	...	1216	1786	2769	16	8	11	1	0	0	2
4	1821	1	1.2	0	13	1	44	0.6	141	2	...	1208	1212	1411	8	2	15	1	1	0	1

5 rows x 21 columns

Q1. WHAT IS MEANT BY REMOVING MISSING VALUES?

Removing missing values means deleting rows that contain null or empty data entries.

Q2.WHY ARE MISSING VALUES REMOVED FROM THE DATASET?

Machine learning algorithms cannot work reliably with missing data. Missing values distort patterns in the dataset and reduce the accuracy of the model.

Q3. WHAT IS MEANT BY REMOVING DUPLICATE ROWS?

Removing duplicate rows means deleting repeated records that appear more than once in the dataset.

Q4. WHY ARE DUPLICATE ROWS REMOVED?

Duplicate records bias the model by giving extra importance to certain examples. This can cause overfitting and poor generalization.

Q5. WHY IS DATA CLEANING IMPORTANT BEFORE MODEL TRAINING?

Data cleaning ensures that the dataset is unbiased, reliable, and suitable for transformations and model training.

● VERIFY FEATURE UNIT:-

Feature unit verification is the process of ensuring that all values within a single feature column are measured using the **same unit**. While different columns may have different units, **a single column must not contain mixed units**, as this can confuse the machine learning model and lead to incorrect learning.

In this project, all numerical feature columns were carefully analyzed by examining their value ranges to confirm unit consistency. Features such as **RAM, internal memory, battery power, screen resolution, mobile weight, screen dimensions, depth, and clock speed** were checked individually.

The analysis confirmed that:

- Each feature column uses **one consistent unit throughout**
- No column contains mixed units (e.g., MB and GB together in the same column)
- All values fall within realistic and expected ranges for mobile phone specifications

Therefore, the dataset was found to be **unit-consistent and reliable**, making it suitable for further preprocessing steps such as conversion, feature engineering, and model training.

CODE:-


```

for col in df.columns:
    print(f"\n==== {col} =====")
    try:
        print(df[col].describe())
    except:
        print("Non-numeric column")

```

OUTPUT:-

```

===== sc_w =====
count    2000.000000
mean      5.767000
std       4.356398
min       0.000000
25%       2.000000
50%       5.000000
75%       9.000000
max      18.000000
Name: sc_w, dtype: float64

===== talk_time =====
count    2000.000000
mean     11.011000
std      5.463955
min       2.000000
25%       6.000000
50%      11.000000
75%      16.000000
max      20.000000
Name: talk_time, dtype: float64

===== three_g =====
count    2000.000000
mean      0.761500
std       0.426273
min       0.000000
25%       1.000000
50%       1.000000
75%       1.000000
max       1.000000
Name: three_g, dtype: float64

===== touch_screen =====
count    2000.000000
mean      0.503000
std       0.500116
min       0.000000
25%       0.000000
50%       1.000000
75%       1.000000
max       1.000000
Name: touch_screen, dtype: float64

===== wifi =====
count    2000.000000
mean      0.507000
std       0.500076
min       0.000000
25%       0.000000
50%       1.000000
75%       1.000000
max       1.000000
Name: wifi, dtype: float64

===== pc =====
count    2000.000000
mean      9.916500
std       6.064315
min       0.000000
25%       5.000000
50%      10.000000
75%      15.000000
max      20.000000
Name: pc, dtype: float64

===== px_height =====
count    2000.000000
mean     645.108000
std      443.780811
min       0.000000
25%      282.750000
50%      564.000000
75%      947.250000
max     1960.000000
Name: px_height, dtype: float64

===== px_width =====
count    2000.000000
mean     1251.515500
std      432.199447
min       500.000000
25%      874.750000
50%     1247.000000
75%     1633.000000
max     1998.000000
Name: px_width, dtype: float64

===== ram =====
count    2000.000000
mean     2124.213000
std     1084.732044
min      256.000000
25%     1207.500000
50%     2146.500000
75%     3064.500000
max     3998.000000
Name: ram, dtype: float64

===== sc_h =====
count    2000.000000
mean      4.213245
std       5.000000
min       0.000000
25%       9.000000
50%      12.000000
75%      16.000000
max      19.000000
Name: sc_h, dtype: float64

===== four_g =====
count    2000.000000
mean      0.521500
std       0.499662
min       0.000000
25%       0.000000
50%       1.000000
75%       1.000000
max       1.000000
Name: four_g, dtype: float64

===== int_memory =====
count    2000.000000
mean     32.046500
std     18.145715
min       2.000000
25%      16.000000
50%      32.000000
75%      48.000000
max      64.000000
Name: int_memory, dtype: float64

===== m_dep =====
count    2000.000000
mean      0.501750
std       0.288416
min       0.100000
25%       0.200000
50%       0.500000
75%       0.800000
max       1.000000
Name: m_dep, dtype: float64

===== mobile_wt =====
count    2000.000000
mean     140.249000
std      35.399655
min       80.000000
25%     109.000000
50%     141.000000
75%     170.000000
max     200.000000
Name: mobile_wt, dtype: float64

===== n_cores =====
count    2000.000000
mean      4.520500
std       2.287837
min       1.000000
25%       3.000000
50%       4.000000
75%       7.000000
max      8.000000
Name: n_cores, dtype: float64

===== battery_power =====
count    2000.000000
mean    1238.518500
std     439.418206
min     501.000000
25%    851.750000
50%    1226.000000
75%    1615.250000
max    1998.000000
Name: battery_power, dtype: float64

===== blue =====
count    2000.0000
mean      0.4950
std       0.5001
min       0.0000
25%       0.0000
50%       0.0000
75%       1.0000
max       1.0000
Name: blue, dtype: float64

===== clock_speed =====
count    2000.000000
mean     1.522250
std      0.816004
min      0.500000
25%      0.700000
50%      1.500000
75%      2.200000
max      3.000000
Name: clock_speed, dtype: float64

===== dual_sim =====
count    2000.000000
mean      0.509500
std       0.500035
min       0.000000
25%       0.000000
50%       1.000000
75%       1.000000
max       1.000000
Name: dual_sim, dtype: float64

===== fc =====
count    2000.000000
mean      4.309500
std      4.341444
min       0.000000
25%       1.000000
50%       3.000000
75%       7.000000
max      19.000000
Name: fc, dtype: float64

```

CHECKING EACH COLUMN UNIT USING VALUE RANGE

I will tell you **what the unit is** and **how I know**.

1. battery_power

Range: 501 → 1998 This perfectly matches **mAh** range of older smartphones. ✓ **Unit is consistent: mAh**

2. ram

Range: 256 → 3998 This is **MB**, because:

- Modern phones in GB would show 1 → 12
- You have values like 256, 1207, 3064 → these are MB numbers

✓**Unit is consistent: MB** You must convert to **GB** later.

3. int_memory

Range: 2 → 64 Phones in this dataset typically had 4GB, 8GB, 16GB, 32GB, 64GB.

✓**Unit is consistent: GB**

No MB values present. No mixed units.

4. px_height

Range: 0 → 1960 0 means missing or low-resolution placeholders.

✓**Unit: pixels**

5. px_width

Range: 500 → 1998 This matches old phone widths.

✓**Unit: pixels**

6. sc_h (screen height in *cm-like arbitrary units*)

Range: 5 → 19 Phones do NOT have 19-inch screens, so this is not inches. This dataset uses **arbitrary scaled screen dimensions**.

✓**Unit: abstract units (dataset-defined)**

7. sc_w

Range: 0 → 18

✓**Same: abstract units.**

8. m_dep

Range: 0.1 → 1.0 Phone thickness around 7–10 mm → 0.7–1.0 cm.

✓Unit: cm

9. mobile_wt

Range: 80 → 200

✓Unit: grams

10. clock_speed

Range: 0.5 → 3.0

✓Unit: GHz

11. price_range (target)

Values 0,1,2,3

✓No units. Only categories.

FINAL — UNIT CONSISTENCY RESULT

Column	Expected Unit	Actual Unit	Mixed Units?
ram	MB	MB	✗ No mixing
int_memory	GB	GB	✗ No mixing
battery_power	mAh	mAh	✗ No mixing
px_height	pixels	pixels	✗ No mixing
px_width	pixels	pixels	✗ No mixing
mobile_wt	grams	grams	✗ No mixing
m_dep	cm	cm	✗ No mixing
sc_h	units	units	✗ No mixing
sc_w	units	units	✗ No mixing
clock_speed	GHz	GHz	✗ No mixing

Q1: WHAT IF WE USE RAM AS MB ONLY WITHOUT CONVERTING IT ? WHY THE DATASET USES RAM IN MB INSTEAD OF GB?

ANSWER: Because this dataset comes from older smartphones (2015 era) where:

256 MB

512 MB
1024 MB
2048 MB
3072 MB
4096 MB

were common values.

GB-level RAM existed but was usually shown in MB in early datasets.

Q2: DOES IT AFFECT MODEL PERFORMANCE?

ANSWER: No — ML models don't care if RAM is in MB or GB, as long as:

Units are consistent

You standardize/normalize later

But MB is a large number (3000+), so converting to GB reduces scale, making graphs and engineering easier.

Q3: WHY YOU CONVERT RAM TO GB?

ANSWER:

1. *More readable----3072 MB → 3 GB*
2. *Better feature engineering Ratios like: ram_gb / int_memory_gb make sense only if both are in the same unit system.*
3. *Scaling becomes smooth Instead of values ~3000, you get values ~3.*
4. *Industry standard representation Everyone uses GB for RAM today.*

● **UNIT CONVERSION FROM MB TO GB:-**

Unit conversion from **Megabytes (MB)** to **Gigabytes (GB)** is the process of changing memory values into a common and standard unit so that all data is consistent and easy to interpret.

Since **1 GB = 1024 MB**, the conversion is done by dividing the MB value by **1024**

- **Overwrite original RAM column**

Code:-

```
df['ram'] = df['ram'] / 1024
```

RESULT:-

- ram becomes GB
- No extra column created

Q1: WHAT IF WE USE RAM AS MB ONLY WITHOUT CONVERTING IT ? WHY THE DATASET USES RAM IN MB INSTEAD OF GB?

ANSWER: Because this dataset comes from older smartphones (2015 era) where:

256 MB

512 MB

1024 MB

2048 MB

3072 MB

4096 MB

were common values.

GB-level RAM existed but was usually shown in MB in early datasets.

Q2: DOES IT AFFECT MODEL PERFORMANCE?

ANSWER: No — ML models don't care if RAM is in MB or GB, as long as:

Units are consistent

You standardize/normalize later

But MB is a large number (3000+), so converting to GB reduces scale, making graphs and engineering easier.

Q3: WHY YOU CONVERT RAM TO GB?

ANSWER:

1. More readable----3072 MB → 3 GB

2. *Better feature engineering Ratios like: $\text{ram_gb} / \text{int_memory_gb}$ make sense only if both are in the same unit system.*
3. *Scaling becomes smooth Instead of values ~ 3000 , you get values ~ 3 .*
4. *Industry standard representation: Everyone uses GB for RAM today.*

● EXPLORATORY DATA ANALYSIS (EDA)

EDA (Exploratory Data Analysis) is the process of analyzing and visualizing data to understand its structure, patterns, and problems before applying machine-learning models.

- **Dataset Info:**

The dataset information is checked to understand the number of rows, columns, data types, and overall structure of the data.

- **Missing Values Check:**

Missing values are analyzed to ensure data completeness, as missing entries can negatively affect model performance.

- **Statistical Summary:**

Descriptive statistics such as mean, minimum, maximum, and standard deviation are examined to understand the data distribution and value ranges.

- **Target Distribution (Price Range):**

The distribution of the target variable (price range) is analyzed to check class balance among low, mid, and high-priced mobiles.

- **Correlation Heatmap:**

A correlation heatmap is used to identify relationships between features and detect highly correlated variables that influence the price range.

- **Outlier summary:**

An outlier summary shows the number of outliers detected in each feature of the dataset. Outliers are data points that lie

far away from the normal range of values and can affect model performance.

- **Feature distribution box plot:**

A feature distribution box plot is a statistical visualization used to show how the values of a feature are distributed across different categories of the target variable.

- **Feature Distribution Plot (Histogram with KDE):**

Important features such as battery power, RAM, pixel height, pixel width, and internal memory are compared with price range to understand their impact on mobile pricing.

- **Scatter plot:**

A scatter plot is a graphical representation used to show the relationship between two numerical features by plotting data points on a two-dimensional plane.

Each point in the plot represents a single data sample, where:

- The x-axis represents one feature
- The y-axis represents another feature

CODE:-

```
import matplotlib.pyplot as plt
import seaborn as sns

# Basic info
print("Dataset Info:")
print(df.info())

print("\nStatistical Summary:")
print(df.describe())

print("\nTarget Distribution (price_range):")
print(df['price_range'].value_counts())

# ----- 3. Missing values check -----
print("\nMissing values per column:")
print(df.isna().sum())

# Correlation Heatmap
plt.figure(figsize=(14, 10))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Heatmap')
plt.show()

# ----- 4. Outlier detection (IQR) -----
outlier_count = {}

num_cols = df.select_dtypes(include='number').columns

for col in num_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outlier_count[col] = ((df[col] < lower) | (df[col] > upper)).sum()

outlier_df = pd.DataFrame.from_dict(outlier_count, orient='index', columns=['outlier_count'])
print("\nOutlier summary:")
print(outlier_df.sort_values(by='outlier_count', ascending=False))

# Key features vs price_range
key_features = ['battery_power', 'ram', 'px_height', 'px_width', 'int_memory']

for col in key_features:
    plt.figure(figsize=(8, 4))
    sns.boxplot(x='price_range', y=col, data=df)
    plt.title(f'{col} vs price_range')
    plt.show()

for col in key_features:
    plt.figure(figsize=(8, 4))
    sns.histplot(data=df, x=col, hue='price_range', multiple='stack', kde=True)
    plt.title(f'Distribution of {col} by price_range')
    plt.show()
```

OUTPUT:-

```
... Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 21 columns):
#   Column              Non-Null Count  Dtype
---  -
0   battery_power        2000 non-null   int64
1   blue                 2000 non-null   int64
2   clock_speed          2000 non-null   float64
3   dual_sim             2000 non-null   int64
4   fc                  2000 non-null   int64
5   four_g              2000 non-null   int64
6   int_memory           2000 non-null   int64
7   m_dep               2000 non-null   float64
8   mobile_wt           2000 non-null   int64
9   n_cores              2000 non-null   int64
10  pc                   2000 non-null   int64
11  px_height            2000 non-null   int64
12  px_width             2000 non-null   int64
13  ram                  2000 non-null   int64
14  sc_h                 2000 non-null   int64
15  sc_w                 2000 non-null   int64
16  talk_time            2000 non-null   int64
17  three_g              2000 non-null   int64
18  touch_screen         2000 non-null   int64
19  wifi                 2000 non-null   int64
20  price_range          2000 non-null   int64
dtypes: float64(2), int64(19)
memory usage: 328.3 KB
None

Statistical Summary:
count    battery_power    blue    clock_speed    dual_sim    fc  \
mean      1238.518500    0.4950    1.522250    0.509500    4.309500
std        439.418206    0.5001    0.816004    0.500035    4.341444
min        501.000000    0.0000    0.500000    0.000000    0.000000
25%        851.750000    0.0000    0.700000    0.000000    1.000000
50%       1226.000000    0.0000    1.500000    1.000000    3.000000
75%       1615.250000    1.0000    2.200000    1.000000    7.000000
max       1998.000000    1.0000    3.000000    1.000000   19.000000

count    four_g    int_memory    m_dep    mobile_wt    n_cores  ...  \
mean     0.521500    32.046500    0.501750    140.249000    4.520500  ...
std      0.499662    18.145715    0.288416    35.399655    2.287837  ...
min      0.000000     2.000000    0.100000    80.000000    1.000000  ...
25%      0.000000    16.000000    0.200000    109.000000    3.000000  ...
50%      1.000000    32.000000    0.500000    141.000000    4.000000  ...
75%      1.000000    48.000000    0.800000    170.000000    7.000000  ...
max      1.000000    64.000000    1.000000    200.000000    8.000000  ...

count    px_height    px_width    ram    sc_h    sc_w  \
mean     645.108000   1251.515500  2124.213000  12.306500  5.767000
std      443.780811    432.199447  1084.732044   4.213245  4.356398
min       0.000000    500.000000   256.000000   5.000000  0.000000
25%      282.750000    874.750000  1207.500000   9.000000  2.000000
50%      564.000000   1247.000000  2146.500000  12.000000  5.000000
75%      947.250000   1633.000000  3064.500000  16.000000  9.000000
max     1960.000000   1998.000000  3998.000000  19.000000  18.000000

count    talk_time    three_g    touch_screen    wifi    price_range
mean     11.011000    0.761500    0.503000    0.507000    1.500000
std       5.463955    0.426273    0.500116    0.500076    1.118314
min       2.000000    0.000000    0.000000    0.000000    0.000000
25%       6.000000    1.000000    0.000000    0.000000    0.750000
50%      11.000000    1.000000    1.000000    1.000000    1.500000
75%      16.000000    1.000000    1.000000    1.000000    2.250000
max      20.000000    1.000000    1.000000    1.000000    3.000000

[8 rows x 21 columns]
```

Target Distribution (price_range):

price_range

1 500

2 500

3 500

0 500

Name: count, dtype: int64

Missing values per column:

battery_power 0

blue 0

clock_speed 0

dual_sim 0

fc 0

four_g 0

int_memory 0

m_dep 0

mobile_wt 0

n_cores 0

pc 0

px_height 0

px_width 0

ram 0

sc_h 0

sc_w 0

talk_time 0

three_g 0

touch_screen 0

wifi 0

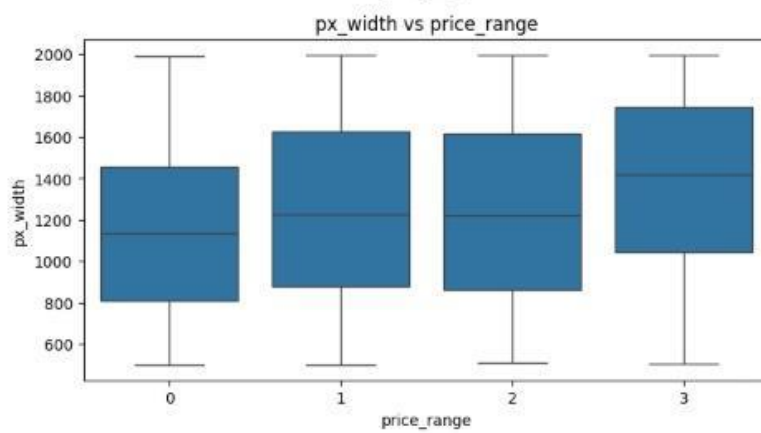
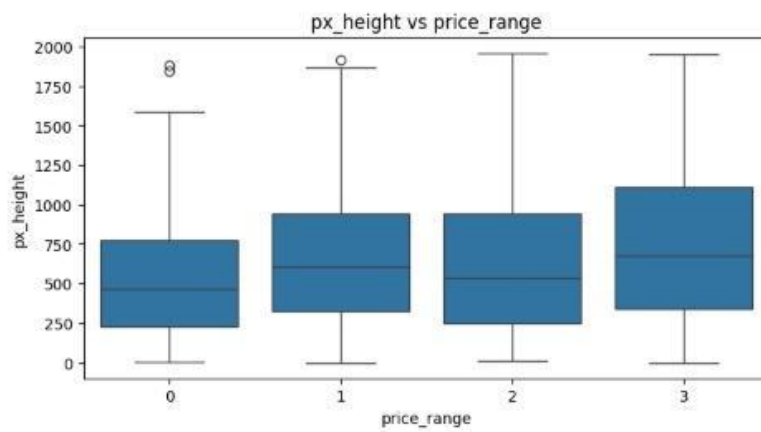
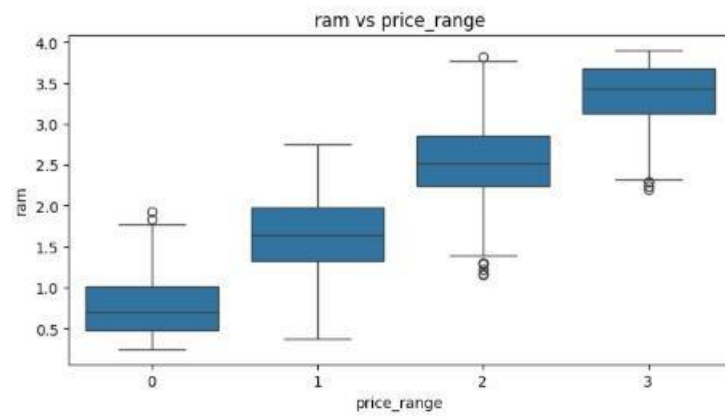
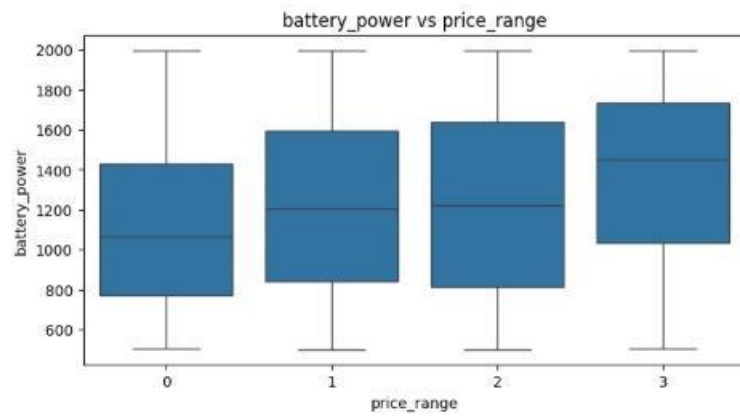
price_range 0

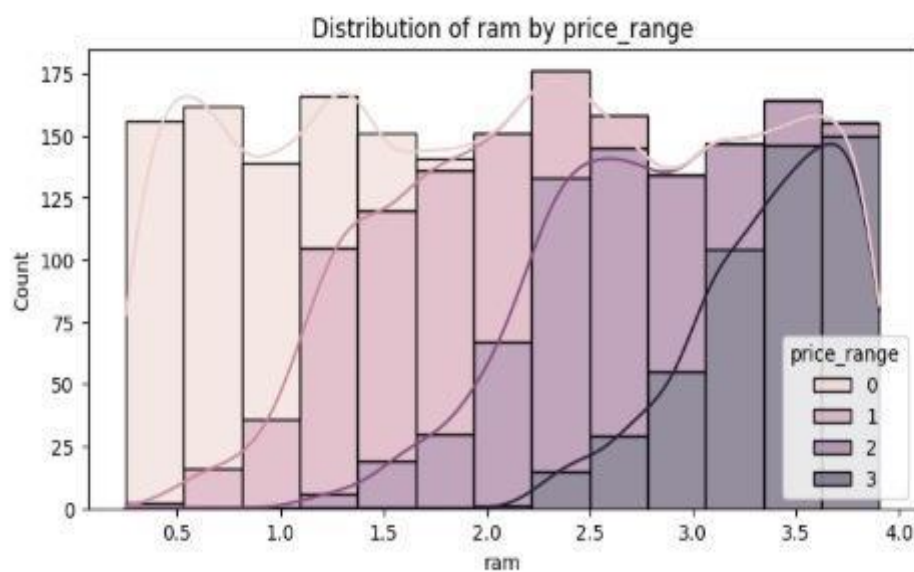
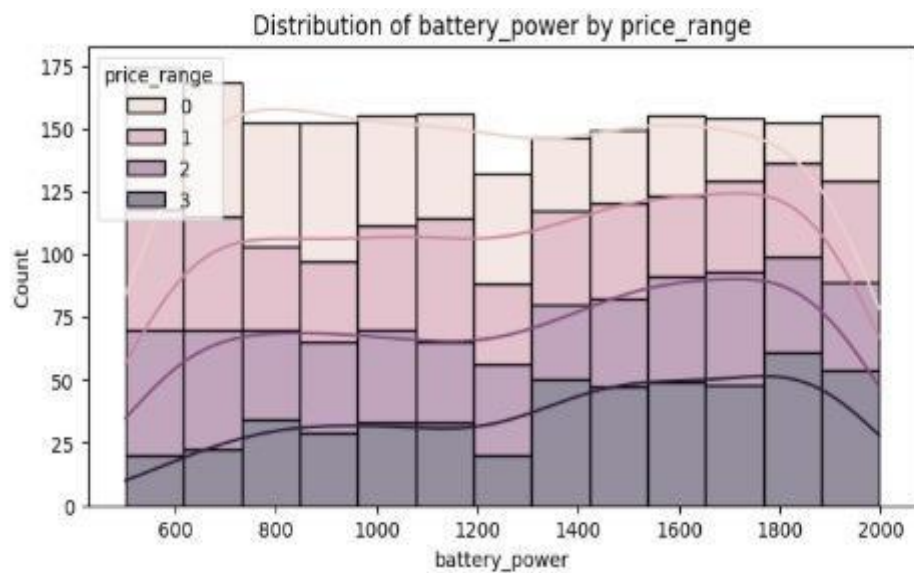
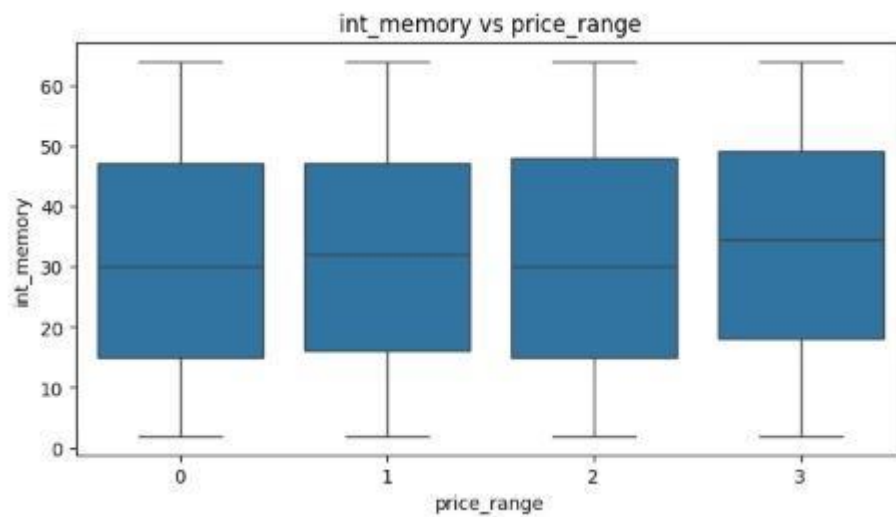
dtype: int64

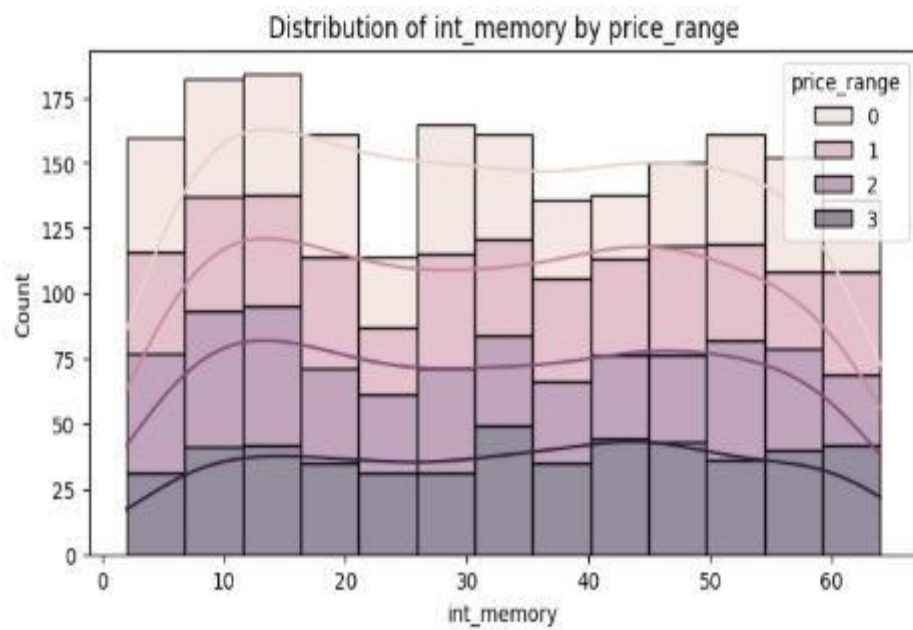
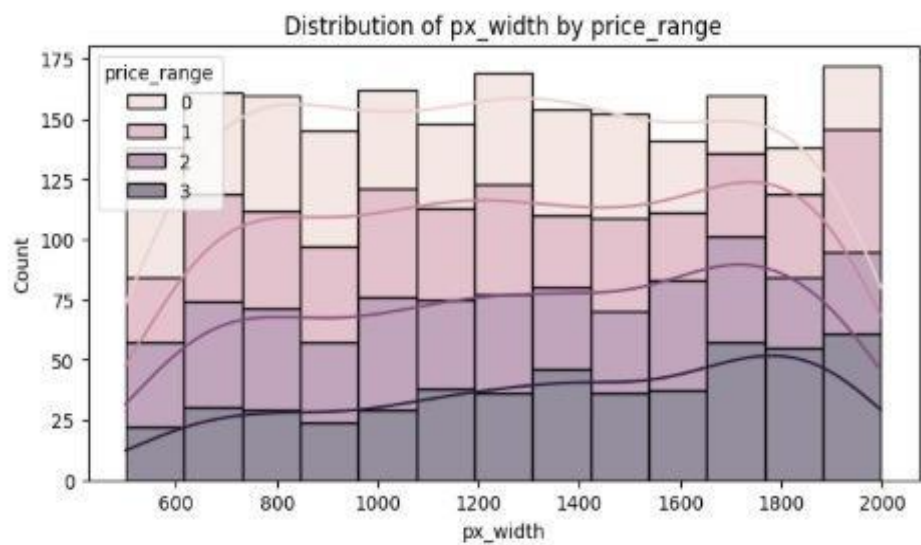
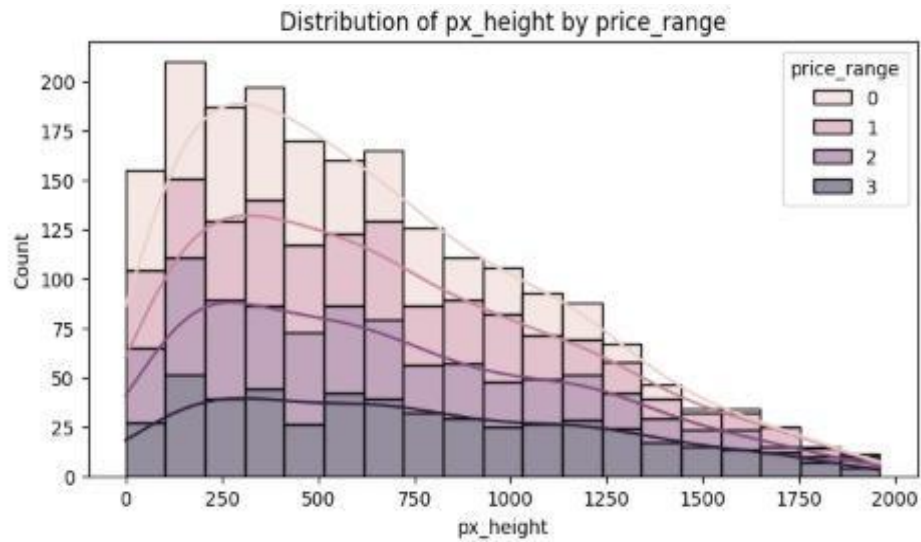
Outlier summary:

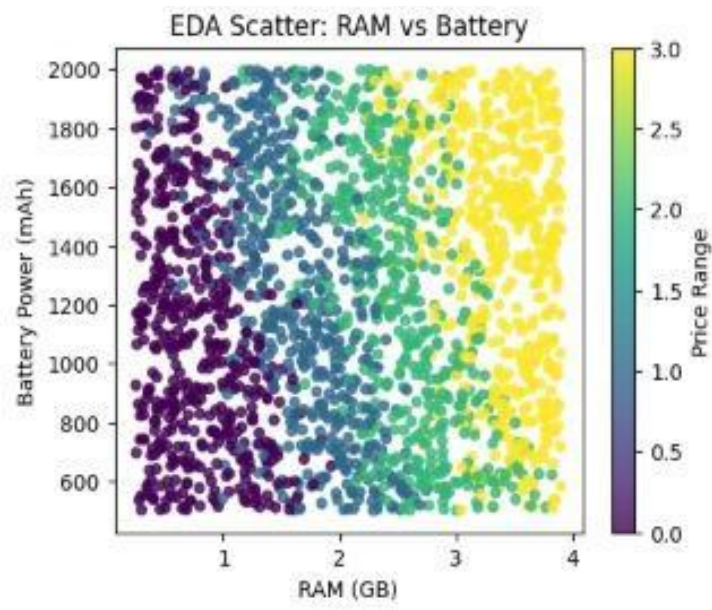
	outlier_count
three_g	477
fc	18

px_height	2
battery_power	0
blue	0
dual_sim	0
clock_speed	0
m_dep	0
four_g	0
mobile_wt	0
n_cores	0
pc	0
int_memory	0
px_width	0
ram	0
sc_w	0
sc_h	0
talk_time	0
touch_screen	0
wifi	0
price_range	0









● FEATURE ENGINEERING:-

Feature Engineering is the process of transforming raw data into meaningful features that improve model performance.

In this project, f, and the target variable (price range) is encoded into numerical labels (0–3). Standardization is applied so that all features are on a similar scale, preventing any single feature from dominating the model.

Overall, feature engineering improves data quality, model accuracy, and learning efficiency.

Q.1 WHAT IT DOES?

- *Combines columns*
- *Transforms values*
- *Creates ratios*
- *Creates interactions*
- *Adds logs, bins, categories*
- *Extracts hidden patterns the model won't see by itself*

Makes the data more meaningful

Q.2 WHY IT MATTERS?

1. *It increases model accuracy more than any other step.*
2. *Example*

*From px_height + px_width → create resolution
From battery / weight → create battery_efficiency*

3. *In one line*

Feature engineering turns raw data into smarter data so your model performs better.

CODE:-

```
import numpy as np
import pandas as pd

# REQUIRE: df loaded & cleaned
assert 'df' in globals(), "DataFrame 'df' not found. Load/clean data first."

# Make a local copy reference (operations inplace on df)
# Ensure numeric types where needed
num_cols = ['ram', 'int_memory', 'px_height', 'px_width', 'sc_h', 'sc_w', 'battery_power', 'mobile_wt']
for c in num_cols:
    if c in df.columns:
        df[c] = pd.to_numeric(df[c], errors='coerce')

# 1) Ensure ram in GB (heuristic)
if 'ram' in df.columns:
    # if median > 50 -> values likely in MB
    if df['ram'].median(skipna=True) > 50:
        df['ram'] = df['ram'] / 1024.0

# 2) Ensure int_memory exists
if 'int_memory' not in df.columns:
    df['int_memory'] = 0.0
else:
    df['int_memory'] = df['int_memory'].fillna(0.0)

# 3) Resolution (total pixels) - requires px_height & px_width
if {'px_height', 'px_width'}.issubset(df.columns):
    df['resolution'] = (df['px_height'].astype(float) * df['px_width'].astype(float)).fillna(0.0)
else:
    df['resolution'] = 0.0

# 4) PPI - only if px_height, px_width and sc_h (screen diagonal in inches) exist
if {'px_height', 'px_width', 'sc_h'}.issubset(df.columns):
    sc_h_safe = df['sc_h'].replace(0, np.nan).fillna(df['sc_h'].median(skipna=True))
    df['ppi'] = np.sqrt(df['px_height'].astype(float)**2 + df['px_width'].astype(float)**2) / sc_h_safe
    df['ppi'] = df['ppi'].fillna(df['ppi'].median(skipna=True))
else:
    # create ppi with zeros so downstream code won't break
    df['ppi'] = 0.0

# 5) Battery / performance combos - guard each column
if 'battery_power' in df.columns and 'mobile_wt' in df.columns:
    wt_safe = df['mobile_wt'].replace(0, np.nan).fillna(df['mobile_wt'].median(skipna=True))
    df['battery_efficiency'] = (df['battery_power'].astype(float) / wt_safe).fillna(0.0)
else:
    df['battery_efficiency'] = 0.0

if 'ram' in df.columns and 'int_memory' in df.columns:
    df['ram_int_ratio'] = (df['ram'].astype(float) / (df['int_memory'].astype(float) + 1e-6)).replace([np.inf, -np.inf], 0).fillna(0.0)
else:
    df['ram_int_ratio'] = 0.0

if 'ram' in df.columns and 'battery_power' in df.columns:
    df['ram_battery_combo'] = (df['ram'].astype(float) * df['battery_power'].astype(float)).fillna(0.0)
else:
    df['ram_battery_combo'] = 0.0

# 6) Res area (resolution * sc_w) if sc_w present
if 'resolution' in df.columns and 'sc_w' in df.columns:
    df['res_area'] = (df['resolution'].astype(float) * df['sc_w'].astype(float)).fillna(0.0)
else:
    df['res_area'] = 0.0

# 7) Binned RAM level (0..3) - safe bins for ram in GB
if 'ram' in df.columns:
    df['ram_level'] = pd.cut(df['ram'].astype(float),
                             bins=[-1, 1, 3, 6, np.inf],
                             labels=[0, 1, 2, 3]).astype(float).fillna(0).astype(int)
else:
    df['ram_level'] = 0

# 8) Log transforms for skewed features (guard negatives/nans)
for col in ['battery_power', 'ram', 'resolution']:
    if col in df.columns:
        df[f'log_{col}'] = np.log1p(df[col].astype(float).clip(lower=0)).fillna(0.0)
    else:
        df[f'log_{col}'] = 0.0

# 9) Drop any temporary helpers if present (none added permanently)
# (No helper columns were added to df permanently)

# 10) Final sanity: fill remaining NaNs in new features with 0
new_features = ['resolution', 'ppi', 'battery_efficiency', 'ram_int_ratio', 'ram_battery_combo', 'res_area', 'ram_level', 'log_battery_power', 'log_ram', 'log_resolution']
for c in new_features:
    if c in df.columns:
        df[c] = df[c].fillna(0.0)

# 11) Summary (concise)
added = [c for c in new_features if c in df.columns]
print("Added features:", added)
print("New shape:", df.shape)
print(df[added].head())
```

OUTPUT :-

```
... Added features: ['resolution', 'ppi', 'battery_efficiency', 'ram_int_ratio', 'ram_battery_combo', 'res_area', 'ram_level', 'log_battery_power', 'log_ram', 'log_resolution']
New shape: (2000, 31)
resolution      ppi      battery_efficiency      ram_int_ratio \
0      15120.0      84.029989      4.478723      0.355608
1      1799140.0      128.488269      7.507353      0.048478
2      2167308.0      193.698774      3.882759      0.062000
3      2171776.0      135.041255      4.694656      0.270410
4      1464096.0      213.900094      12.914894      0.031317

ram_battery_combo      res_area      ram_level      log_battery_power      log_ram \
0      2095.955078      105840.0      1      6.736967      1.249689
1      2623.291992      5397420.0      1      6.929517      1.272380
2      1431.141602      4334616.0      1      6.335054      1.264689
3      1663.022461      17374208.0      1      6.423247      1.309441
4      2509.209961      2928192.0      1      7.507690      0.866230

log_resolution
0      9.623840
1      14.402820
2      14.588997
3      14.591056
4      14.196749
```

● ENCODING THE TARGET VARIABLE

In machine learning, most algorithms require the target variable to be in a numeric format. Since the price range of mobile phones is a categorical variable (Low, Mid, High), it must be converted into numerical labels before model training. This process is known as target encoding.

In this project, the target variable Price Range was encoded using label encoding, where each price category was assigned a unique numerical value. The encoding scheme used is shown below:

Price Category Encoded Value

Low Price 0

Mid Price 1

High Price 2

This encoding allows machine learning models to understand and process the target variable effectively. The encoded values represent class labels rather than numerical magnitude, making them suitable for classification algorithms such as Logistic Regression, Decision Tree, Random Forest, and K-Nearest Neighbors (KNN).

If the target was not encoded (example: "low", "medium", "high"), then:

1. Model would throw an error

ML algorithms cannot train on text labels.

2. Metrics like accuracy, F1 cannot compute

They expect numbers, not strings.

3. Confusion matrix cannot be generated

It needs numeric classes.

4. Train/Test split might work, but training would fail

Split doesn't care about types, but models do.

5. Hyperparameter tuning and cross-validation would break
GridSearchCV expects numeric y-values.

6. Some algorithms assume classes are ordered

If labels were strings, they cannot infer order

```
# Encoding target – safe, ready-to-copy
import pandas as pd
from sklearn.preprocessing import LabelEncoder

assert 'df' in globals() and 'price_range' in df.columns, "DataFrame 'df' with column 'price_range' required."

# Helper: ensure no missing targets
if df['price_range'].isna().any():
    raise ValueError("Missing values found in 'price_range' – handle them before encoding.")

# Case 1: already proper integer labels 0..3
unique_vals = set(pd.Series(df['price_range']).unique().astype(object))
if pd.api.types.is_integer_dtype(df['price_range']) and unique_vals.issubset({0, 1, 2, 3}):
    df['price_range'] = df['price_range'].astype(int)
else:
    # Case 2: string/categorical labels -> LabelEncoder
    if df['price_range'].dtype == object or pd.api.types.is_categorical_dtype(df['price_range']):
        le = LabelEncoder()
        df['price_range'] = le.fit_transform(df['price_range']).astype(int)

    # Case 3: numeric but not 0..3 (continuous or >4 classes) -> quantile bin into 4 classes
    elif pd.api.types.is_numeric_dtype(df['price_range']):
        if df['price_range'].nunique() > 4:
            df['price_range'] = pd.qcut(df['price_range'], 4, labels=False).astype(int)
        else:
            # Map distinct numeric values to 0..(k-1)
            vals = sorted(df['price_range'].unique())
            mapping = {v: i for i, v in enumerate(vals)}
            df['price_range'] = df['price_range'].map(mapping).astype(int)

# Final safety: ensure labels are 0..3 (if fewer classes, they will be in range 0..k-1)
min_label, max_label = int(df['price_range'].min()), int(df['price_range'].max())
if max_label > 3:
    raise ValueError(f"Encoded labels exceed 3 (got {min_label}-{max_label}). Inspect 'price_range' values.")

print("price_range classes distribution:\n", df['price_range'].value_counts().sort_index())
```

OUTPUT:-

```
*** price_range classes distribution:
    price_range
0         500
1         500
2         500
3         500
Name: count, dtype: int64
```

● **DATASET SPLITTING (TRAIN-TEST SPLIT)**

After completing data preprocessing, the dataset is divided into two separate parts known as the training dataset and the testing dataset. This step is essential to evaluate how effectively the machine learning models can classify mobile phone price ranges.

In this project, the dataset contains mobile specifications such as RAM, Storage, and Processor Score, along with the target variable Price Range (Low, Mid, High). The dataset is typically split using a 70:30 or 80:20 ratio, where:

- 70–80% of the data is used for training
- 20–30% of the data is used for testing

Training Set (80%)

- Used to train the machine learning models
- Helps the model learn patterns from input features
- Used to fit model parameters and weights

Testing Set (20%)

- Used to evaluate the model on unseen data
- Helps measure the accuracy of predictions
- Checks the real performance of the model
- Helps in detecting and avoiding overfitting

- This approach ensures a fair and reliable evaluation of the machine learning models before final selection.

CODE:-

```
# data split
from sklearn.model_selection import train_test_split

# Split features and target
X = df.drop(columns=['price_range'])
y = df['price_range']

# 80% train, 20% test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

print("Train shape :", X_train.shape)
print("Test shape  :", X_test.shape)
```

OUTPUT:-

```
Train shape : (1600, 30)
Test shape  : (400, 30)
```

Q.1 **WHY WE SPLIT?**

Because if the model is tested on the same data it was trained on, accuracy will be fake and meaningless.

In short:

Train set → model learns Test set → model is judged This ensures a fair and realistic evaluation.

❖ STANDARDIZATION:-

Standardization is a feature scaling technique used to transform data so that each feature has a **mean of 0** and a **standard deviation of 1**. This helps bring all features onto a common scale without changing their relative differences.

The standardization formula is:

$$Z = \frac{(x - \mu)}{\sigma}$$

Standardization is important because it ensures that no single feature dominates the learning process due to larger numerical values, leading to better and more stable model performance.

Q.1 **WHAT IT DOES?**

*It puts all features on the **same scale**, so no feature dominates due to larger values.*

Example:

- *RAM = 4*
- *battery = 1500*

Model will think battery is more important unless standardized.

After standardization:

- *RAM → ~0*
- *battery → ~0*

Both become equal scale.

Q.2 **WHY IT'S NEEDED?**

Algorithms like:

- *SVM*
- *Logistic regression*

- Linear regression
- KNN
- Neural networks

perform MUCH better when data is standardized.

In short

Standardization = **scale all features to same range (0 mean, 1 std)** for better model training.

CODE:-

```
#standardization
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

# Fit only on training data
scaler.fit(X_train)

# Transform both datasets
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Standardization complete.")
print("X_train_scaled:", X_train_scaled.shape)
print("X_test_scaled :", X_test_scaled.shape)

'''Standardization code = Z-score scaling code.

Your features become:

mean = 0

std = 1

This is Z-score.'''
```

OUTPUT:-

```
Standardization complete.  
X_train_scaled: (1600, 30)  
X_test_scaled : (400, 30)  
'Standardization code = Z-score scaling code.\n\nYour features become:\n\nmean = 0\n\nstd = 1\n\nThis is Z-score.'
```

You must NOT use both. Use only ONE: Standardization OR Normalization. Never both together.

Reason (straight point-to-point):

1. ***Standardization (Z-score)*** makes mean=0, std=1.
2. ***Normalization (Min-Max)*** makes values 0→1.
3. *If you apply normalization after standardization → **you completely destroy the standardization you just did.***

Effect: – Your models get confused. – Scaling becomes meaningless. – Accuracy becomes inconsistent. – Pipelines break.

Industry rule: For ML models → choose ONE scaling step.

*For your models (Logistic, Linear classifier, KNN): **Standardization is the correct choice.***

Decision Tree / Random Forest do not need scaling at all.

TRAIN ALL FIVE MODEL

In this step, five different machine learning models were trained to classify the mobile price range. These models include **Linear Regression, Logistic Regression, Decision Tree, Random Forest, and K-Nearest Neighbors (KNN)**. Training multiple models helps compare their performance, understand different learning behaviors, and select the best-performing model. Each model learns patterns from the training data and is later evaluated on the test data to determine accuracy and reliability

CODE:-

```
import joblib
import numpy as np
from sklearn.linear_model import SGDClassifier, LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# choose data (scaled preferred)
if 'X_train_scaled' in globals() and 'X_test_scaled' in globals():
    Xtr = X_train_scaled
    Xte = X_test_scaled
else:
    Xtr = X_train.values if hasattr(X_train, "values") else X_train
    Xte = X_test.values if hasattr(X_test, "values") else X_test

# ensure y are numpy arrays
ytr = y_train.values if hasattr(y_train, "values") else y_train
yte = y_test.values if hasattr(y_test, "values") else y_test

model_results = {}
```

```

# -----
# Train: Linear Classifier (SGDClassifier - linear)
# -----
print("\n" + "="*30)
print("Training: Linear Classifier (SGDClassifier - linear)")
print("="*30)

linear_clf = SGDClassifier(loss="squared_hinge", penalty="l2", random_state=42, max_iter=5000, tol=1e-4)
linear_clf.fit(Xtr, ytr)
pred_linear = linear_clf.predict(Xte)

acc_linear = accuracy_score(yte, pred_linear)
print("Linear Classifier Accuracy:", round(acc_linear,4))
print("Confusion Matrix:\n", confusion_matrix(yte, pred_linear))
print("Classification Report:\n", classification_report(yte, pred_linear, zero_division=0))

model_results['LinearClassifier'] = {'model': linear_clf, 'accuracy': acc_linear}
joblib.dump(linear_clf, "model_linear_clf.pkl")

# -----
# Train: Logistic Regression (classification)
# -----
print("\n" + "="*30)
print("Training: Logistic Regression")
print("="*30)

logreg = LogisticRegression(max_iter=5000, random_state=42)
logreg.fit(Xtr, ytr)
pred_logreg = logreg.predict(Xte)

acc_logreg = accuracy_score(yte, pred_logreg)
print("Logistic Regression Accuracy:", round(acc_logreg,4))
print("Confusion Matrix:\n", confusion_matrix(yte, pred_logreg))
print("Classification Report:\n", classification_report(yte, pred_logreg, zero_division=0))

model_results['LogisticRegression'] = {'model': logreg, 'accuracy': acc_logreg}
joblib.dump(logreg, "model_logistic_regression.pkl")

# -----
# Train: Decision Tree Classifier
# -----
print("\n" + "="*30)
print("Training: Decision Tree Classifier")
print("="*30)

dt = DecisionTreeClassifier(random_state=42)
dt.fit(Xtr, ytr)
pred_dt = dt.predict(Xte)

acc_dt = accuracy_score(yte, pred_dt)
print("Decision Tree Accuracy:", round(acc_dt,4))
print("Confusion Matrix:\n", confusion_matrix(yte, pred_dt))
print("Classification Report:\n", classification_report(yte, pred_dt, zero_division=0))

model_results['DecisionTree'] = {'model': dt, 'accuracy': acc_dt}
joblib.dump(dt, "model_decision_tree.pkl")

# -----
# Train: Random Forest Classifier
# -----
print("\n" + "="*30)
print("Training: Random Forest Classifier")
print("="*30)

rf = RandomForestClassifier(n_estimators=200, random_state=42, n_jobs=-1)
rf.fit(Xtr, ytr)
pred_rf = rf.predict(Xte)

acc_rf = accuracy_score(yte, pred_rf)
print("Random Forest Accuracy:", round(acc_rf,4))
print("Confusion Matrix:\n", confusion_matrix(yte, pred_rf))
print("Classification Report:\n", classification_report(yte, pred_rf, zero_division=0))

model_results['RandomForest'] = {'model': rf, 'accuracy': acc_rf}
joblib.dump(rf, "model_random_forest.pkl")

```

```

# -----
# Train: K-Nearest Neighbors (KNN)
# -----
print("\n" + "="*30)
print("Training: K-Nearest Neighbors (KNN)")
print("="*30)

knn = KNeighborsClassifier(n_neighbors=5, n_jobs=-1)
knn.fit(Xtr, ytr)
pred_knn = knn.predict(Xte)

acc_knn = accuracy_score(yte, pred_knn)
print("KNN Accuracy:", round(acc_knn,4))
print("Confusion Matrix:\n", confusion_matrix(yte, pred_knn))
print("Classification Report:\n", classification_report(yte, pred_knn, zero_division=0))

model_results['KNN'] = {'model': knn, 'accuracy': acc_knn}
joblib.dump(knn, "model_knn.pkl")

# -----
# Summary: Best model by accuracy on test set
# -----
print("\n" + "="*30)
best_name = max(model_results, key=lambda k: model_results[k]['accuracy'])
best_acc = model_results[best_name]['accuracy']
print(f"BEST MODEL (by test accuracy): {best_name} with accuracy = {round(best_acc,4)}")
print("="*30)

```

OUTPUT:-

```

...
=====
Training: Linear Classifier (SGDClassifier - linear)
=====
Linear Classifier Accuracy: 0.915
Confusion Matrix:
[[98  2  0  0]
 [ 2 83 15  0]
 [ 0 12 88  0]
 [ 0  0  3 97]]
Classification Report:
      precision    recall  f1-score   support

     0       0.98      0.98      0.98        100
     1       0.86      0.83      0.84        100
     2       0.83      0.88      0.85        100
     3       1.00      0.97      0.98        100

 accuracy          0.92
 macro avg          0.92
weighted avg          0.92

=====
Training: Logistic Regression
=====
Logistic Regression Accuracy: 0.9525
Confusion Matrix:
[[97  3  0  0]
 [ 4 93  3  0]
 [ 0  2 94  4]
 [ 0  0  3 97]]
Classification Report:
      precision    recall  f1-score   support

     0       0.96      0.97      0.97        100
     1       0.95      0.93      0.94        100
     2       0.94      0.94      0.94        100
     3       0.96      0.97      0.97        100

 accuracy          0.95
 macro avg          0.95
weighted avg          0.95

```

```

=====
Training: Decision Tree Classifier
=====
Decision Tree Accuracy: 0.855
Confusion Matrix:
[[94  6  0  0]
 [ 8 80 12  0]
 [ 0 12 79  9]
 [ 0  0 11 89]]
Classification Report:
              precision    recall  f1-score   support

     0       0.92      0.94      0.93       100
     1       0.82      0.80      0.81       100
     2       0.77      0.79      0.78       100
     3       0.91      0.89      0.90       100

 accuracy         0.85
 macro avg        0.86
 weighted avg     0.86

```

```

=====
Training: Random Forest Classifier
=====
Random Forest Accuracy: 0.9375
Confusion Matrix:
[[99  1  0  0]
 [ 2 92  6  0]
 [ 0  7 89  4]
 [ 0  0  5 95]]
Classification Report:
              precision    recall  f1-score   support

     0       0.98      0.99      0.99       100
     1       0.92      0.92      0.92       100
     2       0.89      0.89      0.89       100
     3       0.96      0.95      0.95       100

 accuracy         0.94
 macro avg        0.94
 weighted avg     0.94

```

```

Training: Random Forest Classifier
=====
Random Forest Accuracy: 0.9375
Confusion Matrix:
[[99  1  0  0]
 [ 2 92  6  0]
 [ 0  7 89  4]
 [ 0  0  5 95]]
Classification Report:
              precision    recall  f1-score   support

     0       0.98      0.99      0.99       100
     1       0.92      0.92      0.92       100
     2       0.89      0.89      0.89       100
     3       0.96      0.95      0.95       100

 accuracy         0.94
 macro avg        0.94
 weighted avg     0.94

```

```

=====
Training: K-Nearest Neighbors (KNN)
=====
KNN Accuracy: 0.725
Confusion Matrix:
[[88 12  0  0]
 [13 70 16  1]
 [ 2 31 56 11]
 [ 0  2 22 76]]
Classification Report:
              precision    recall  f1-score   support

     0       0.85      0.88      0.87       100
     1       0.61      0.70      0.65       100
     2       0.60      0.56      0.58       100
     3       0.86      0.76      0.81       100

 accuracy         0.72
 macro avg        0.73
 weighted avg     0.73

```

```

=====
BEST MODEL (by test accuracy): LogisticRegression with accuracy = 0.9525
=====

```

A EXPLANATION OF CODE :-

1. Importing Required Libraries

The code begins by importing essential libraries such as:

- scikit-learn classifiers (SGDClassifier, Logistic Regression, Decision Tree, Random Forest, KNN)*
- Evaluation metrics like accuracy score, confusion matrix, and classification report*
- joblib for saving trained models*

These libraries support model training, evaluation, and storage.

2. Selecting Input Data

The code checks whether standardized datasets (X_train_scaled, X_test_scaled) are available. If present, standardized data is used because many ML algorithms perform better when features are on the same scale. Otherwise, the original datasets are used.

The target variables (y_train, y_test) are converted into NumPy arrays to ensure compatibility with all models.

3. Model Results Storage

A dictionary named model_results is created to store:

- The trained model*
- Its corresponding accuracy score*

This allows easy comparison between all models.

Training Individual Models

a) Linear Classifier (SGDClassifier)

A linear classification model trained using stochastic gradient descent. It is fast and used as a baseline model. Accuracy, confusion

matrix, and classification report are generated, and the model is saved.

b) Logistic Regression (Classification)

A probabilistic classification algorithm suitable for multi-class problems. The model learns decision boundaries between price categories and shows strong performance after standardization.

c) Decision Tree Classifier

A tree-based model that splits data based on feature values. It is easy to interpret but may overfit the data.

d) Random Forest Classifier

An ensemble model that combines multiple decision trees. It improves accuracy and reduces overfitting compared to a single decision tree.

e) K-Nearest Neighbors (KNN)

A distance-based model that classifies a mobile phone based on the majority class of its nearest neighbors.

4. Model Evaluation

For each model:

- Predictions are made on the test dataset*
- Performance is evaluated using accuracy score*
- Confusion matrix and classification report are printed*
- The trained model is saved using joblib*

5. Best Model Selection

Finally, the model with the highest test accuracy is identified automatically and printed as the best performing model for mobile price range classification.

HYPERPARAMETER

TUNING

AHyperparameter tuning is the process of selecting the best set of model parameters (such as learning rate, depth, or number of neighbors) to improve model performance and accuracy.

A model has hyperparameters (knobs you control), like:

- Decision Tree → max_depth, min_samples_split
- Random Forest → n_estimators, max_features
- KNN → n_neighbors
- SVM → C, gamma
- Logistic Regression → C, penalty

Hyperparameter tuning is an optimization process used to improve the performance of a machine learning model.

Hyperparameters are predefined control values that decide **how the model learns**, for example depth of a tree, number of neighbors in KNN, or regularization strength in Logistic Regression.

Choosing incorrect hyperparameter values can lead to **underfitting or overfitting**, reducing model accuracy.

Hyperparameter tuning systematically tests multiple combinations of these parameters instead of relying on trial and error.

Cross-validation is applied during tuning to evaluate each combination on different data splits, ensuring stable and reliable performance.

The set of hyperparameters that produces the **best cross-validation accuracy** is selected as the final configuration.

CODE:-

```
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import SGDClassifier, LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
import joblib
import numpy as np

assert 'X_train' in globals() and 'y_train' in globals(), "Run train_test_split first."

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
n_jobs = -1
verbose = 1

best_estimators = {}
cv_results = {}

# -----
# 1) Linear classifier (SGDClassifier used as linear classifier)
# -----
pipe_linear = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", SGDClassifier(random_state=42, max_iter=5000, tol=1e-4))
])
param_grid_linear = {
    "clf__loss": ["log", "hinge", "squared_hinge"], # log -> logistic, hinge -> SVM-like
    "clf__penalty": ["l2", "l1", "elasticnet"],
    "clf__alpha": [1e-4, 1e-3, 1e-2],
    "clf__l1_ratio": [0.15, 0.5] # only used if elasticnet
}
gs_linear = GridSearchCV(pipe_linear, param_grid_linear, cv=cv, scoring="accuracy",
                          n_jobs=n_jobs, verbose=verbose, refit=True)
gs_linear.fit(X_train, y_train)
best_estimators['LinearClassifier'] = gs_linear.best_estimator_
cv_results['LinearClassifier'] = gs_linear.best_score_
print("LinearClassifier best:", gs_linear.best_params_, round(gs_linear.best_score_,4))
joblib.dump(gs_linear.best_estimator_, "best_linear_clf.pkl")

# -----
# 2) Logistic Regression
# -----
pipe_log = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(max_iter=5000, random_state=42))
])
param_grid_log = {
    "clf__C": [0.01, 0.1, 1, 10],
    "clf__penalty": ["l2"], # keep simple/stable
    "clf__solver": ["lbfgs", "saga"]
}
gs_log = GridSearchCV(pipe_log, param_grid_log, cv=cv, scoring="accuracy",
                       n_jobs=n_jobs, verbose=verbose, refit=True)
gs_log.fit(X_train, y_train)
best_estimators['LogisticRegression'] = gs_log.best_estimator_
cv_results['LogisticRegression'] = gs_log.best_score_
print("LogisticRegression best:", gs_log.best_params_, round(gs_log.best_score_,4))
joblib.dump(gs_log.best_estimator_, "best_logistic_regression.pkl")
```



```

# -----
# 3) Decision Tree
# -----
dt = DecisionTreeClassifier(random_state=42)
param_grid_dt = {
    "max_depth": [None, 5, 10, 15, 25],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4],
    "criterion": ["gini", "entropy"]
}
gs_dt = GridSearchCV(dt, param_grid_dt, cv=cv, scoring="accuracy",
                     n_jobs=n_jobs, verbose=verbose, refit=True)
gs_dt.fit(X_train, y_train)
best_estimators['DecisionTree'] = gs_dt.best_estimator_
cv_results['DecisionTree'] = gs_dt.best_score_
print("DecisionTree best:", gs_dt.best_params_, round(gs_dt.best_score_,4))
joblib.dump(gs_dt.best_estimator_, "best_decision_tree.pkl")

# -----
# 4) Random Forest
# -----
rf = RandomForestClassifier(random_state=42, n_jobs=1) # n_jobs handled by GridSearchCV
param_grid_rf = {
    "n_estimators": [100, 200],
    "max_depth": [None, 10, 20],
    "max_features": ["sqrt", "log2"],
    "min_samples_split": [2, 5]
}
gs_rf = GridSearchCV(rf, param_grid_rf, cv=cv, scoring="accuracy",
                     n_jobs=n_jobs, verbose=verbose, refit=True)
gs_rf.fit(X_train, y_train)
best_estimators['RandomForest'] = gs_rf.best_estimator_
cv_results['RandomForest'] = gs_rf.best_score_
print("RandomForest best:", gs_rf.best_params_, round(gs_rf.best_score_,4))
joblib.dump(gs_rf.best_estimator_, "best_random_forest.pkl")

```

```

# -----
# 5) K-Nearest Neighbors (pipeline with scaler)
# -----
pipe_knn = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsClassifier())
])
param_grid_knn = {
    "knn_n_neighbors": [3, 5, 7, 9],
    "knn_weights": ["uniform", "distance"],
    "knn_p": [1, 2] # Manhattan (1) or Euclidean (2)
}
gs_knn = GridSearchCV(pipe_knn, param_grid_knn, cv=cv, scoring="accuracy",
                      n_jobs=n_jobs, verbose=verbose, refit=True)
gs_knn.fit(X_train, y_train)
best_estimators['KNN'] = gs_knn.best_estimator_
cv_results['KNN'] = gs_knn.best_score_
print("KNN best:", gs_knn.best_params_, round(gs_knn.best_score_,4))
joblib.dump(gs_knn.best_estimator_, "best_knn.pkl")

# -----
# Summary: best by CV score
# -----
best_name = max(cv_results, key=cv_results.get)
best_score = cv_results[best_name]
print("\nBEST (by CV accuracy):", best_name, round(best_score,4))

# Save summary results
joblib.dump(best_estimators, "best_estimators_all_models.pkl")
print("Saved all best estimators and individual best models to disk.")

```

OUTPUT:-

```
LinearClassifier best: {'clf_alpha': 0.001, 'clf_l1_ratio': 0.15, 'clf_loss': 'hinge', 'clf_penalty': 'l1'} 0.9181
Fitting 5 folds for each of 8 candidates, totalling 40 fits
LogisticRegression best: {'clf_C': 10, 'clf_penalty': 'l2', 'clf_solver': 'lbfgs'} 0.95
Fitting 5 folds for each of 90 candidates, totalling 450 fits
DecisionTree best: {'criterion': 'entropy', 'max_depth': None, 'min_samples_leaf': 2, 'min_samples_split': 2} 0.865
Fitting 5 folds for each of 24 candidates, totalling 120 fits
RandomForest best: {'max_depth': 10, 'max_features': 'sqrt', 'min_samples_split': 2, 'n_estimators': 100} 0.9062
Fitting 5 folds for each of 16 candidates, totalling 80 fits
KNN best: {'knn_n_neighbors': 9, 'knn_p': 1, 'knn_weights': 'distance'} 0.785

BEST (by CV accuracy): LogisticRegression 0.95
Saved all best estimators and individual best models to disk.
```

COMPARISON AND EVALUATION

● CROSS-VALIDATION COMPARISON

Cross-validation is a technique used to evaluate the reliability and stability of a machine learning model by testing it on different subsets of the training data.

Working of K-Fold Cross-Validation:

- The training dataset is divided into K equal parts (folds), usually 5 or 10.
- The model is trained on K-1 folds.
- The remaining 1 fold is used for testing.
- This process is repeated K times, so each fold is used once as the test set.
- The final performance is calculated by averaging all K accuracy scores.

Q.1 HOW IT WORKS (K-FOLD CV EXAMPLE):

1. Split training data into **K folds** (usually 5 or 10).
2. Train on K-1 folds.
3. Test on the remaining 1 fold.
4. Repeat this K times (each fold becomes test once).
5. Average all scores → **CV accuracy**.

Q.2 WHY IT MATTERS:

- Reduces overfitting risk

- Gives a more reliable accuracy
- Makes hyper-tuning fair
- Ensures model works well on unseen data

CODE:-

```
# ===== Cross-Validation Comparison for All 5 Tuned Models =====
from sklearn.model_selection import cross_validate, StratifiedKFold
import pandas as pd

assert 'best_estimators' in globals(), "Run hyperparameter tuning first."

# Same CV folds for all models
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Scoring metrics (use string names)
scoring = {
    "accuracy": "accuracy",
    "precision": "precision_macro",
    "recall": "recall_macro",
    "f1": "f1_macro"
}

cv_summary = {}

print("\n===== CROSS-VALIDATION COMPARISON (ALL 5 MODELS) =====")

for model_name, model in best_estimators.items():
    print(f"\nEvaluating Model: {model_name}")

    scores = cross_validate(
        model,
        X_train,
        y_train,
        cv=cv,
        scoring=scoring,
        n_jobs=-1,
        verbose=0,
        return_train_score=False
    )

    cv_summary[model_name] = {
        "Accuracy": scores["test_accuracy"].mean(),
        "Precision": scores["test_precision"].mean(),
        "Recall": scores["test_recall"].mean(),
        "F1 Score": scores["test_f1"].mean()
    }

# Display results sorted by F1 score
cv_df = pd.DataFrame(cv_summary).T
cv_df_sorted = cv_df.sort_values(by="F1 Score", ascending=False)

print("\n===== CV RESULTS (Sorted by F1 Score) =====")
print(cv_df_sorted)

best_model_name = cv_df_sorted.index[0]
print(f"\nBEST MODEL (by CV F1): {best_model_name}")
```

OUTPUT:-

```
***
===== CROSS-VALIDATION COMPARISON (ALL 5 MODELS) =====

Evaluating Model: LinearClassifier
Evaluating Model: LogisticRegression
Evaluating Model: DecisionTree
Evaluating Model: RandomForest
Evaluating Model: KNN

===== CV RESULTS (Sorted by F1 Score) =====
              Accuracy  Precision   Recall   F1 Score
LogisticRegression  0.950000    0.951089  0.950000  0.949928
LinearClassifier    0.918125    0.919231  0.918125  0.918233
RandomForest       0.906250    0.907588  0.906250  0.906392
DecisionTree       0.865000    0.867476  0.865000  0.865179
KNN                0.785000    0.790832  0.785000  0.786556

BEST MODEL (by CV F1): LogisticRegression
```

● FINALE TEST EVALUATION:-

Final Test Evaluation is the **last and most important step** in the machine learning pipeline. In this step, the **best tuned model** is evaluated using the test dataset (X_{test}, y_{test}), which contains data that the model has **never seen before**. This evaluation shows the true performance of the model in real-world conditions.

The model performance is analyzed using the following metrics:

1. CONFUSION MATRIX

- A confusion matrix is a **4×4 table** because the target variable has **four classes (0, 1, 2, 3)**.
- Rows represent the **actual class**, and columns represent the **predicted class**.
- Diagonal values show **correct predictions**, while off-diagonal values indicate **misclassifications**.

The confusion matrix helps in:

- Identifying which price ranges are predicted correctly.

- Understanding where the model gets confused between classes.
- Analyzing patterns of wrong predictions (e.g., class 2 predicted as class 3).

2. VISUAL REPRESENTATION OF CONFUSION MATRIX:

Visual Representation of Confusion Matrix

A **confusion matrix** is a visual tool used to evaluate the performance of a classification model by comparing **actual class labels** with **predicted class labels**.

In the visual representation:

- **Rows** represent the actual classes
- **Columns** represent the predicted classes
- Each cell shows the **number of correct and incorrect predictions**

The diagonal elements indicate **correct classifications**, while the off-diagonal elements represent **misclassifications**.

This visualization helps in understanding:

- Model accuracy
- Class-wise performance
- Types of errors made by the model

3. CLASSIFICATION REPORT

The classification report provides a **detailed performance analysis** for each class and includes:

- **Precision:** How accurate the model's predictions are for each class.
- **Recall:** How well the model identifies all samples of each class.

- **F1 Score:** The balance between precision and recall.
- **Macro Average:** Average performance treating all classes equally.
- **Weighted Average:** Average performance considering class distribution.

4.TEST ACCURACY

- Test accuracy measures **how many predictions were correct** out of the total test samples.
- It gives an overall idea of the model's correctness on unseen data.

Formula:

$$\text{Test Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Test Samples}}$$

Final Test Evaluation ensures that the selected model is accurate, reliable, and generalizes well to new data. It confirms whether the model is suitable for predicting mobile price ranges in real-world scenarios.

CONFUSION MATRIX:-

A confusion matrix is a table that shows **how many predictions your model got correct and incorrect for each class**.

Since your problem has **4 classes (0,1,2,3)**, the confusion matrix is a **4×4 grid**.

Q.1 WHY IT'S IMPORTANT

A confusion matrix tells you:

1. **Which classes the model predicts correctly**
2. **Where the model gets confused**
3. **Which specific classes are difficult**

For example:

- *If many 1's are predicted as 2 → model is confused between those two classes*
- *If zeros have perfect diagonal → model predicts class 0 very well*

In your result

Logistic Regression's confusion matrix:

```
[[100  0  0  0]
 [ 1 96  3  0]
 [ 0  3 96  1]
 [ 0  0  2 98]]
```

Meaning:

- *Class 0 → perfect*
- *Class 1 → a few predicted as 2*
- *Class 2 → a few predicted as 1*

- *Class 3 - almost perfect*

*This proves Logistic Regression is your **best model**.*

CONFUSION MATRICES AND CLASSIFICATION REPORT FOR ALL 5 TUNED MODEL

AConfusion matrices and classification reports were used to evaluate all five tuned models. The confusion matrix shows correct and incorrect predictions for each price range class (0–3), helping identify misclassification patterns. The classification report provides precision, recall, F1-score, and overall averages, allowing a fair comparison of model performance beyond accuracy.

CODE:-

```
from sklearn.metrics import confusion_matrix, classification_re
import pandas as pd

assert 'best_estimators' in globals(), "Run hypertuning first."

print("\n===== CONFUSION MATRIX FOR ALL MODELS =====")

all_confusions = {}

for name, model in best_estimators.items():
    print("\n" + "="*40)
    print(f"Model: {name}")
    print("="*40)

    # Classification report
    print("\nClassification Report:")
    print(classification_report(y_test, y_pred, zero_division=0

    # Store confusion matrix for later use
    all_confusions[name] = cm

# Optional: Convert all confusion matrices into a readable dict
print("\n\n===== STORED CONFUSION MATRICES (dictionary format)
for name, cm in all_confusions.items():
    print(f"\n{name}:\n{cm}")
```

OUTPUT:-

===== CONFUSION MATRIX FOR ALL MODELS =====

=====

Model: LinearClassifier

Test Accuracy: 0.9275

Confusion Matrix:

```
[[99  1  0  0]
 [ 2 85 13  0]
 [ 0 10 90  0]
 [ 0  0  3 97]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.98	0.99	0.99	100
1	0.89	0.85	0.87	100
2	0.85	0.90	0.87	100
3	1.00	0.97	0.98	100
accuracy			0.93	400
macro avg	0.93	0.93	0.93	400
weighted avg	0.93	0.93	0.93	400

=====

Model: LogisticRegression

Test Accuracy: 0.975

Confusion Matrix:

```
[[100  0  0  0]
 [  1 96  3  0]
 [  0  3 96  1]
 [  0  0  2 98]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.99	1.00	1.00	100
1	0.97	0.96	0.96	100
2	0.95	0.96	0.96	100
3	0.99	0.98	0.98	100
accuracy			0.97	400
macro avg	0.98	0.97	0.97	400
weighted avg	0.98	0.97	0.97	400

```
=====
Model: DecisionTree
=====
Test Accuracy: 0.885
```

Confusion Matrix:

```
[[96  4  0  0]
 [ 2 88 10  0]
 [ 0 16 79  5]
 [ 0  0  9 91]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.98	0.96	0.97	100
1	0.81	0.88	0.85	100
2	0.81	0.79	0.80	100
3	0.95	0.91	0.93	100
accuracy			0.89	400
macro avg	0.89	0.89	0.89	400
weighted avg	0.89	0.89	0.89	400

```
=====
Model: RandomForest
=====
Test Accuracy: 0.9375
```

Confusion Matrix:

```
[[98  2  0  0]
 [ 3 93  4  0]
 [ 0  6 90  4]
 [ 0  0  6 94]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.97	0.98	0.98	100
1	0.92	0.93	0.93	100
2	0.90	0.90	0.90	100
3	0.96	0.94	0.95	100
accuracy			0.94	400
macro avg	0.94	0.94	0.94	400
weighted avg	0.94	0.94	0.94	400

...

=====
Model: KNN
=====

Test Accuracy: 0.825

Confusion Matrix:

```
[[95  5  0  0]
 [ 3 78 19  0]
 [ 0 22 71  7]
 [ 0  0 14 86]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.97	0.95	0.96	100
1	0.74	0.78	0.76	100
2	0.68	0.71	0.70	100
3	0.92	0.86	0.89	100
accuracy			0.82	400
macro avg	0.83	0.82	0.83	400
weighted avg	0.83	0.82	0.83	400

===== STORED CONFUSION MATRICES (dictionary format) =====

LinearClassifier:

```
[[99  1  0  0]
 [ 2 85 13  0]
 [ 0 10 90  0]
 [ 0  0  3 97]]
```

LogisticRegression:

```
[[100  0  0  0]
 [  1 96  3  0]
 [  0  3 96  1]
 [  0  0  2 98]]
```

DecisionTree:

```
[[96  4  0  0]
 [ 2 88 10  0]
 [ 0 16 79  5]
 [ 0  0  9 91]]
```

RandomForest:

```
[[98  2  0  0]
 [ 3 93  4  0]
 [ 0  6 90  4]
 [ 0  0  6 94]]
```

KNN:

```
[[95  5  0  0]
 [ 3 78 19  0]
 [ 0 22 71  7]
 [ 0  0 14 86]]
```

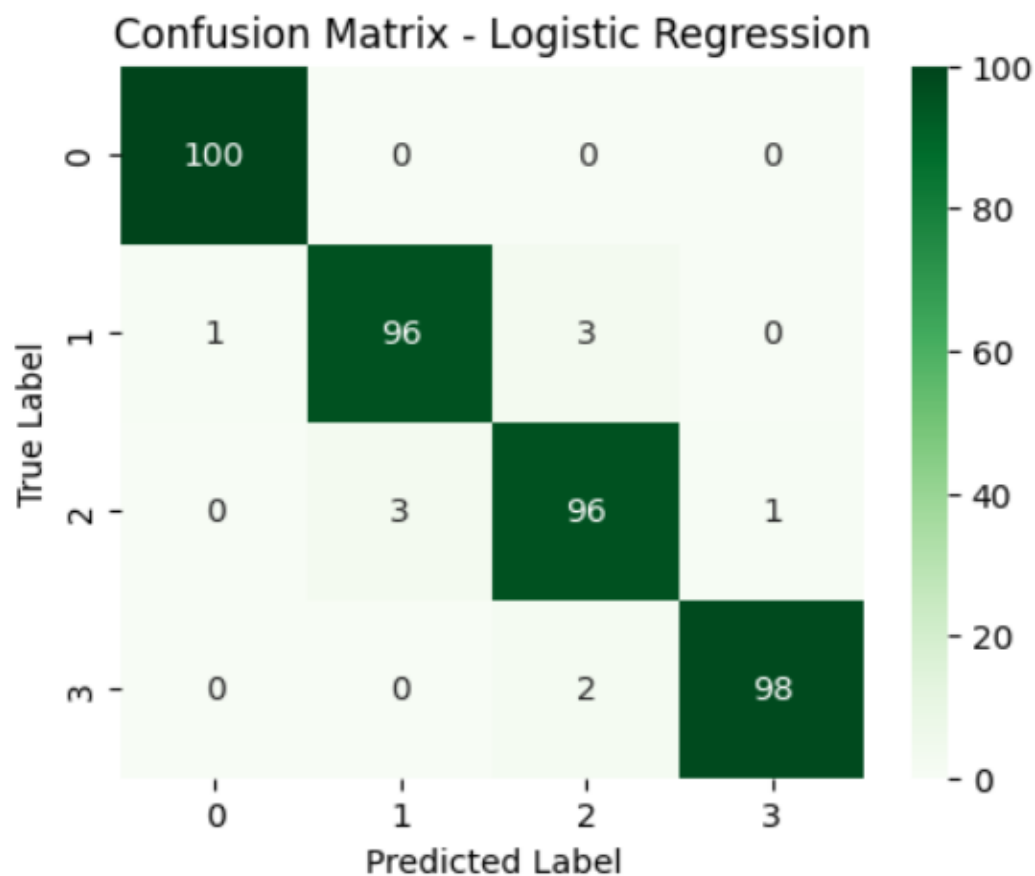
● CONFUSION MATRIX VISUAL REPRESENTATION

1. LOGISTIC REGRESSION

```
#Logistic Regression (BEST MODEL)
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

model = best_estimators["LogisticRegression"]
y_pred = model.predict(X_test)
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Greens",
            xticklabels=[0,1,2,3], yticklabels=[0,1,2,3])
plt.title("Confusion Matrix - Logistic Regression")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()
```

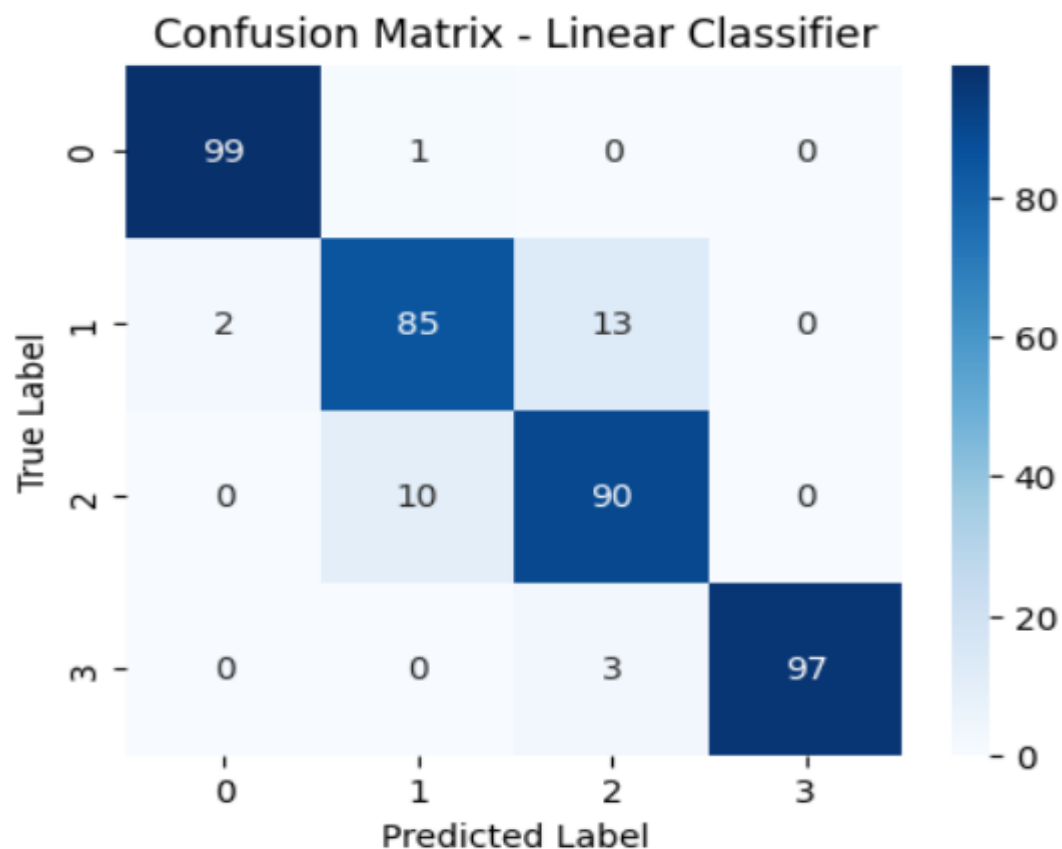


2 LINEAR CLASSIFIER

```
#linear classifier
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

model = best_estimators["LinearClassifier"]
y_pred = model.predict(X_test)
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=[0,1,2,3], yticklabels=[0,1,2,3])
plt.title("Confusion Matrix - Linear Classifier")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()
```

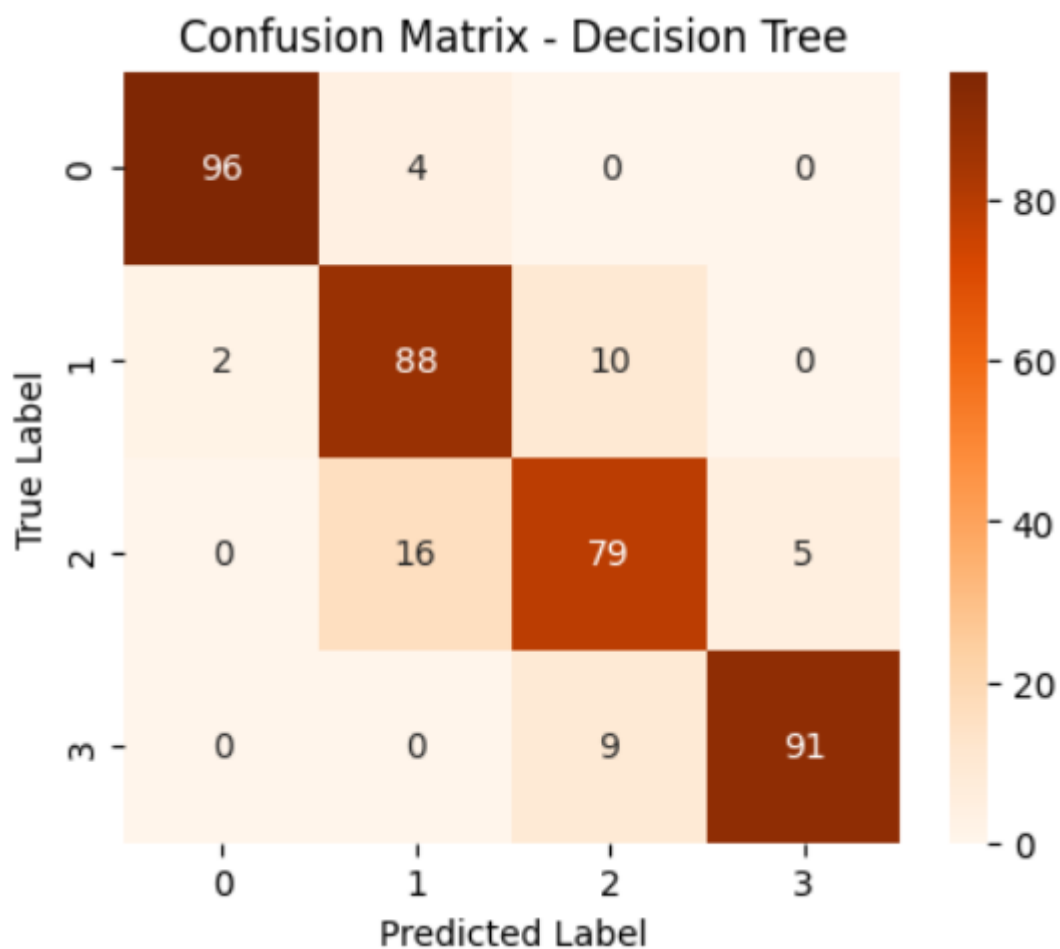


3.DECISION TREE

```
#decision tree
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

model = best_estimators["DecisionTree"]
y_pred = model.predict(X_test)
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Oranges",
            xticklabels=[0,1,2,3], yticklabels=[0,1,2,3])
plt.title("Confusion Matrix - Decision Tree")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()
```

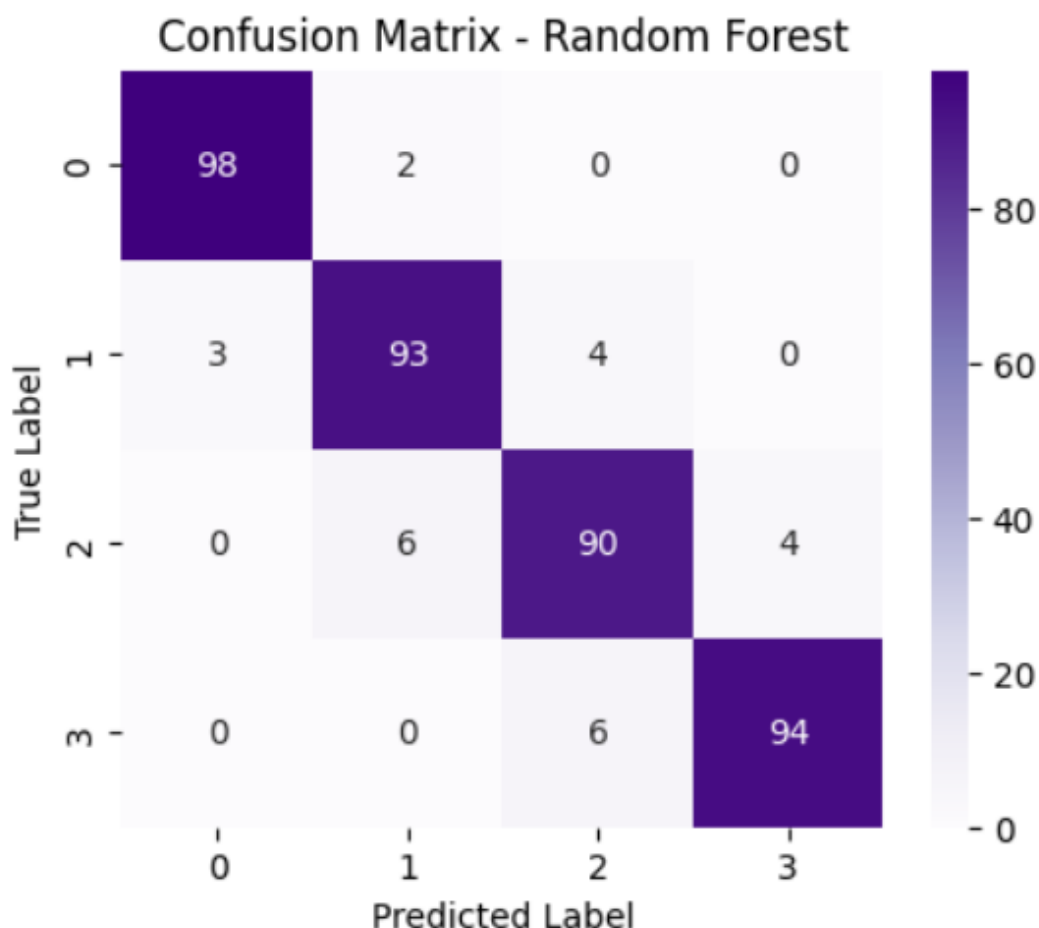


4 RANDOM FOREST

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

model = best_estimators["RandomForest"]
y_pred = model.predict(X_test)
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Purples",
            xticklabels=[0,1,2,3], yticklabels=[0,1,2,3])
plt.title("Confusion Matrix - Random Forest")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()
```



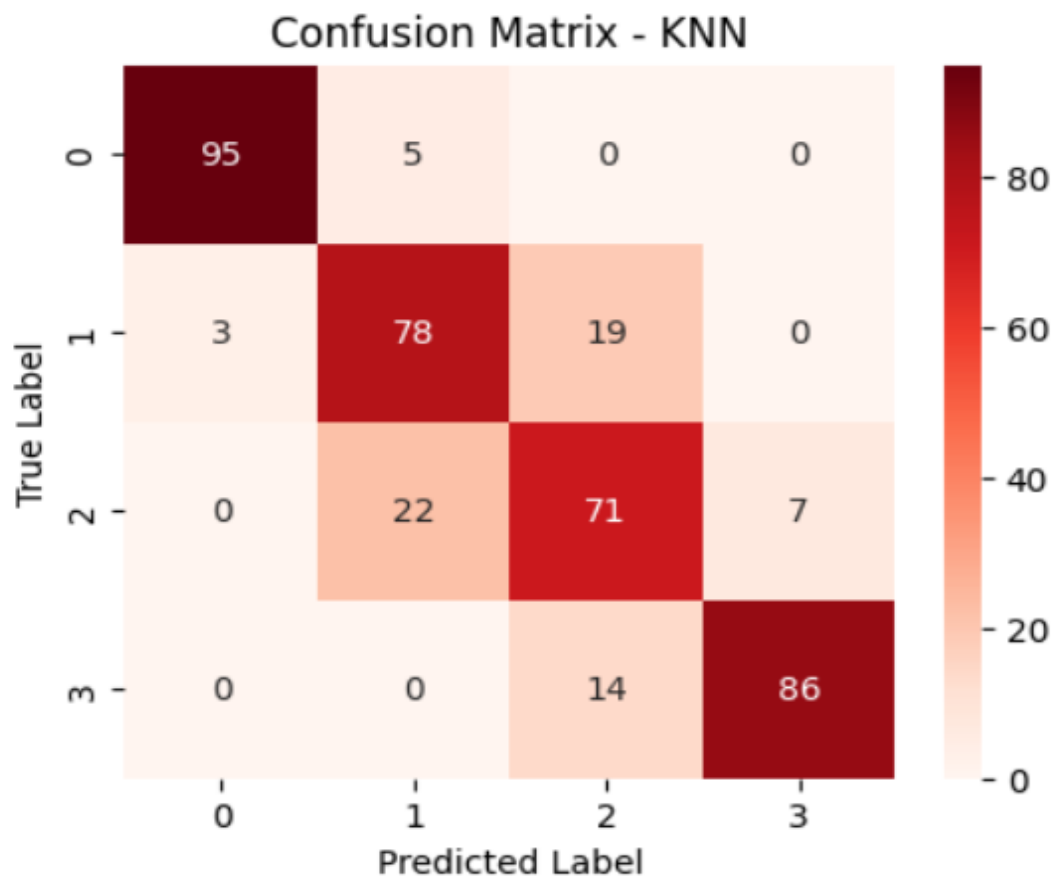
5 KNN

#KNN

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
```

```
model = best_estimators["KNN"]
y_pred = model.predict(X_test)
cm = confusion_matrix(y_test, y_pred)
```

```
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Reds",
            xticklabels=[0,1,2,3], yticklabels=[0,1,2,3])
plt.title("Confusion Matrix - KNN")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()
```



CLASSIFICATION REPORT

A classification report evaluates a model's performance using four key metrics for each class (0-3):

- **Precision:** Percentage of correct predictions made for a class.
- **Recall:** Percentage of actual class samples correctly identified by the model.
- **F1-score:** Balanced measure of precision and recall.
- **Support:** Number of test samples in each class.

It also provides **macro** and **weighted averages**, which help assess overall model performance, especially when comparing different models.

CODE:-

```
# ===== CLASSIFICATION REPORT FOR BEST MODEL =====
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix

# best model name and model
print(f"\n=== BEST MODEL SELECTED AFTER CV: {best_model_name} ===")

best_model = best_estimators[best_model_name]

# predict on test set
y_pred = best_model.predict(X_test)

print("\n=== FINAL TEST ACCURACY ===")
print(round(accuracy_score(y_test, y_pred), 4))

print("\n=== CONFUSION MATRIX ===")
print(confusion_matrix(y_test, y_pred))

print("\n=== CLASSIFICATION REPORT ===")
print(classification_report(y_test, y_pred, zero_division=0))

print("\nThis is the final evaluation of the BEST model:", best_model_name)
```

OUTPUT:-

```
=== BEST MODEL SELECTED AFTER CV: LogisticRegression ===
```

```
=== FINAL TEST ACCURACY ===  
0.975
```

```
=== CONFUSION MATRIX ===
```

```
[[100  0  0  0]  
 [  1 96  3  0]  
 [  0  3 96  1]  
 [  0  0  2 98]]
```

```
=== CLASSIFICATION REPORT ===
```

	precision	recall	f1-score	support
0	0.99	1.00	1.00	100
1	0.97	0.96	0.96	100
2	0.95	0.96	0.96	100
3	0.99	0.98	0.98	100
accuracy			0.97	400
macro avg	0.98	0.97	0.97	400
weighted avg	0.98	0.97	0.97	400

This is the final evaluation of the BEST model: LogisticRegression

TEST ACCURACY

```
● # ===== FINAL TEST ACCURACY COMPARISON FOR ALL 5 TUNED MODELS =====  
# This code evaluates every tuned model on X_test and prints:  
# - Model name  
# - Test accuracy  
# - Best model overall  
  
from sklearn.metrics import accuracy_score  
  
assert 'best_estimators' in globals(), "Run hypertuning first."  
  
test_scores = {}  
  
print("\n===== FINAL TEST ACCURACY OF ALL MODELS =====")  
  
for name, model in best_estimators.items():  
    y_pred = model.predict(X_test)  
    acc = accuracy_score(y_test, y_pred)  
  
    test_scores[name] = acc  
    print(f"{name}: {round(acc, 4)}")  
  
# Find best model  
best_model_test = max(test_scores, key=test_scores.get)  
best_accuracy_test = test_scores[best_model_test]  
  
print("\n===== BEST MODEL ON TEST SET =====")  
print(f"Model: {best_model_test}")  
print(f"Accuracy: {round(best_accuracy_test, 4)}")
```

outlier:-

```
===== FINAL TEST ACCURACY OF ALL MODELS =====
```

```
LinearClassifier: 0.9275
```

```
LogisticRegression: 0.975
```

```
DecisionTree: 0.885
```

```
RandomForest: 0.9375
```

```
KNN: 0.825
```

```
===== BEST MODEL ON TEST SET =====
```

```
Model: LogisticRegression
```

```
Accuracy: 0.975
```

FEATURE IMPORTANCE:-

Feature importance is a technique used in machine learning to measure **how much each input feature contributes to a model's prediction**.

It indicates which features have the **greatest influence** on the target variable and which features have **little or no impact** on the model's output.

It shows:

- Which features strongly affect the price range
- Which features have low or no impact
- How the model prioritizes RAM, battery, resolution, PPI, storage, etc.

Q.1 WHICH MODELS GIVE FEATURE IMPORTANCE?

Yes (supports feature importance):

- *Decision Tree*
- *Random Forest*
- *Logistic Regression (via coefficients)*
- *Linear Classifier (via weights)*

Q.2 Why Feature Importance is Important

Feature importance helps to:

- *Interpret and explain the model's behavior*
- *Identify the most influential features*
- *Reduce model complexity by removing less important features*
- *Improve model performance and generalization*
- *Validate feature engineering decisions*

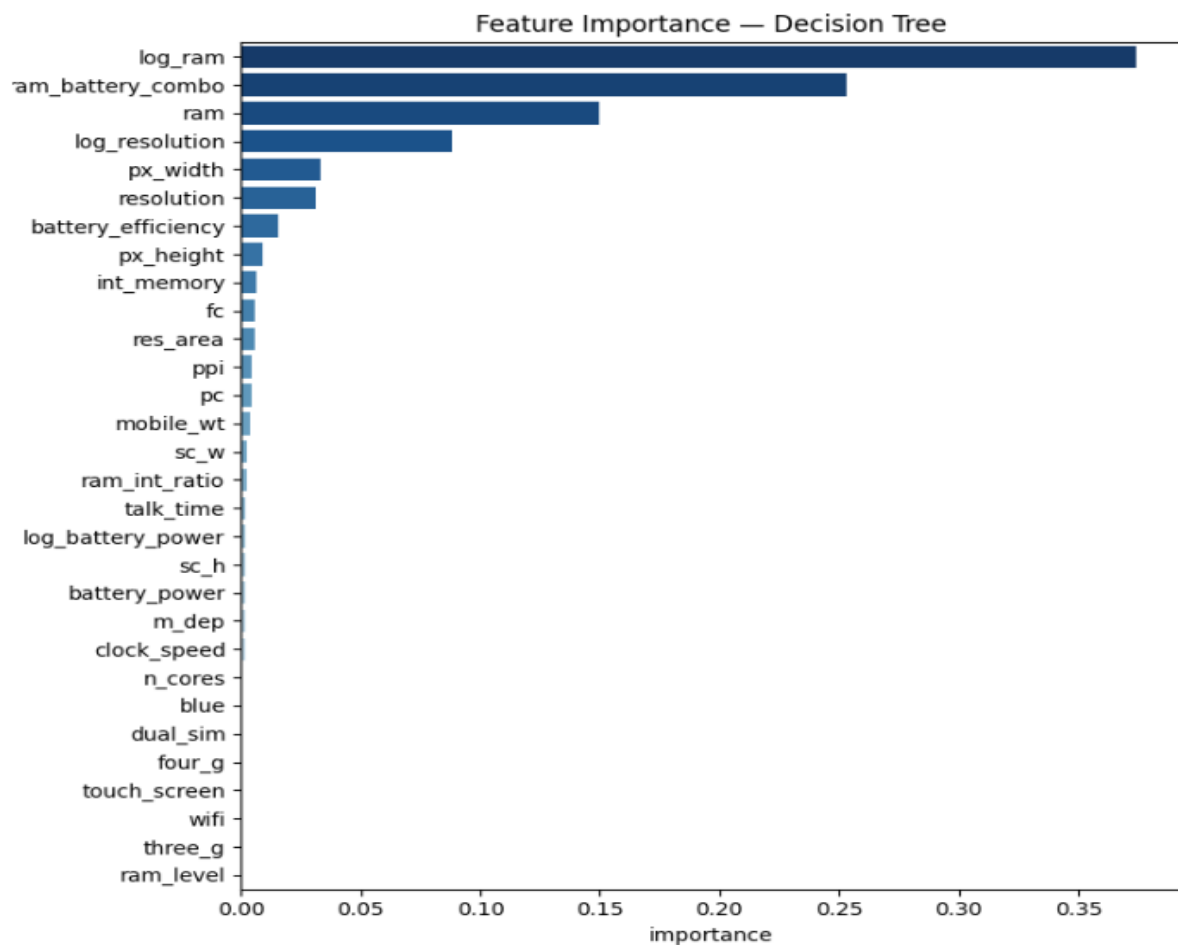
DECISION TREE:-

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

dt_model = best_estimators["DecisionTree"]

imp = dt_model.feature_importances_
df_dt = pd.DataFrame({
    "feature": X_train.columns,
    "importance": imp
}).sort_values("importance", ascending=False)
|
print(df_dt)

plt.figure(figsize=(8, max(4, 0.25 * len(df_dt))))
sns.barplot(x=df_dt["importance"], y=df_dt["feature"], palette="Blues_r")
plt.title("Feature Importance — Decision Tree")
plt.tight_layout()
plt.show()
```



RANDOM FOREST:-

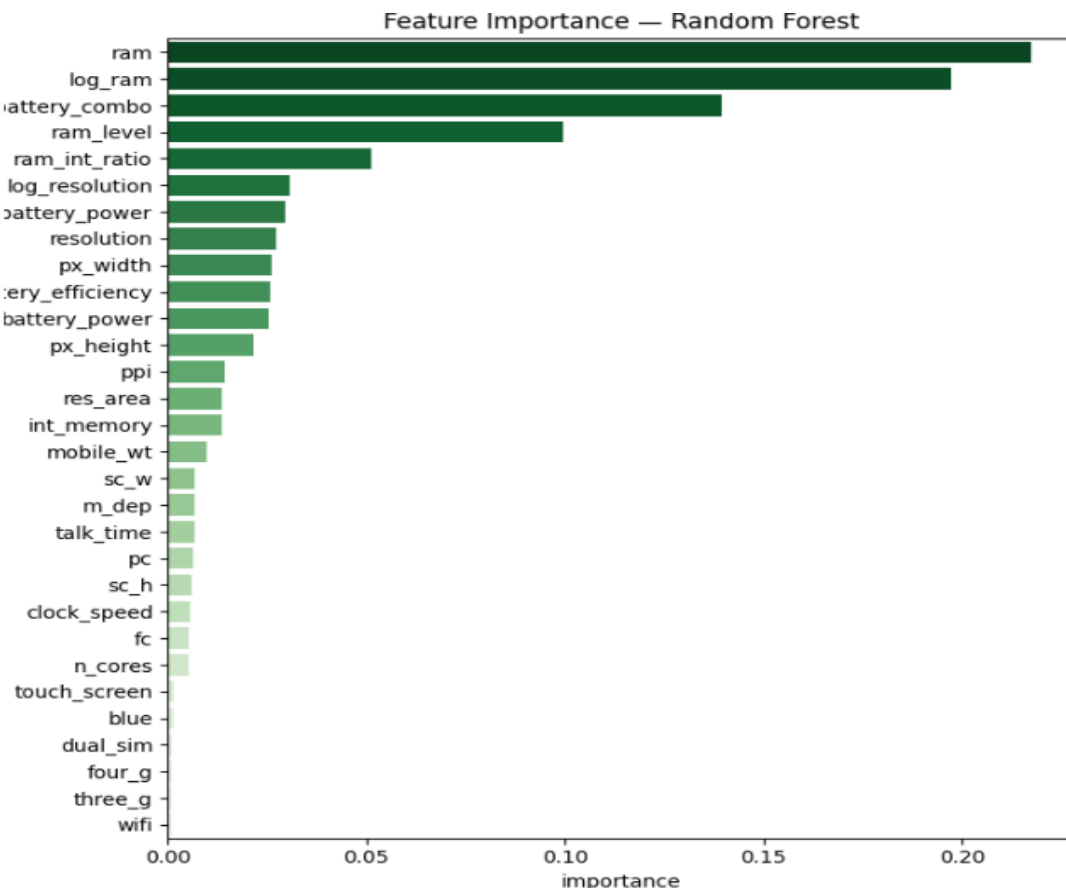
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

rf_model = best_estimators["RandomForest"]

imp = rf_model.feature_importances_
df_rf = pd.DataFrame({
    "feature": X_train.columns,
    "importance": imp
}).sort_values("importance", ascending=False)

print(df_rf)

plt.figure(figsize=(8, max(4, 0.25 * len(df_rf))))
sns.barplot(x=df_rf["importance"], y=df_rf["feature"], palette="Greens_r")
plt.title("Feature Importance - Random Forest")
plt.tight_layout()
plt.show()
```



LOGISTIC REGRESSION

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.pipeline import Pipeline

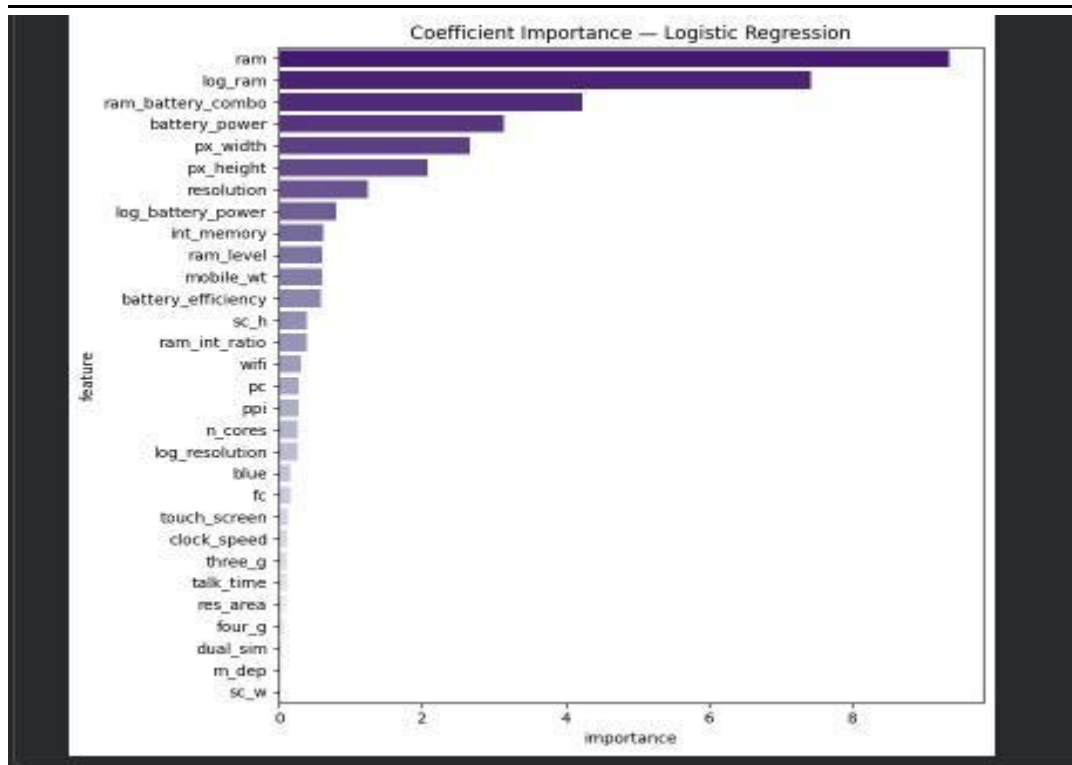
# extract core estimator (last step of pipeline)
log_est = best_estimators["LogisticRegression"]
log_core = log_est.steps[-1][1] if isinstance(log_est, Pipeline) else log_est

coef = np.abs(log_core.coef_).mean(axis=0)

df_log = pd.DataFrame({
    "feature": X_train.columns,
    "importance": coef
}).sort_values("importance", ascending=False)

print(df_log)

plt.figure(figsize=(8, max(4, 0.25 * len(df_log))))
sns.barplot(x=df_log["importance"], y=df_log["feature"], palette="Purples_r")
plt.title("Coefficient Importance – Logistic Regression")
plt.tight_layout()
plt.show()
```



LINEAR CLASSIFIER SGD

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.pipeline import Pipeline

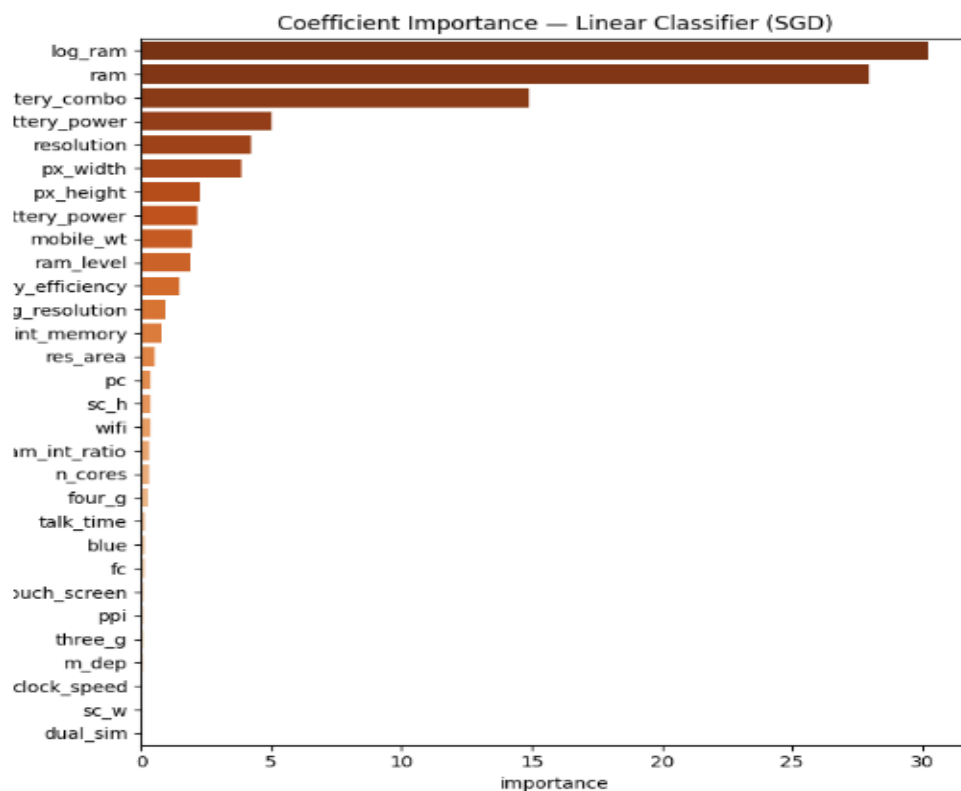
lin_est = best_estimators["LinearClassifier"]
lin_core = lin_est.steps[-1][1] if isinstance(lin_est, Pipeline) else lin_est

coef = np.abs(lin_core.coef_).mean(axis=0)

df_lin = pd.DataFrame({
    "feature": X_train.columns,
    "importance": coef
}).sort_values("importance", ascending=False)

print(df_lin)

plt.figure(figsize=(8, max(4, 0.25 * len(df_lin))))
sns.barplot(x=df_lin["importance"], y=df_lin["feature"], palette="Oranges_r")
plt.title("Coefficient Importance — Linear Classifier (SGD)")
plt.tight_layout()
plt.show()
```



KNN:-

```
from sklearn.inspection import permutation_importance
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

assert 'best_estimators' in globals(), "Run hypertuning first."
assert "KNN" in best_estimators, "KNN not found in best_estimators."
assert hasattr(X_train, "columns"), "X_train must be a DataFrame."

knn_model = best_estimators["KNN"]

print("Computing permutation importance for KNN...")

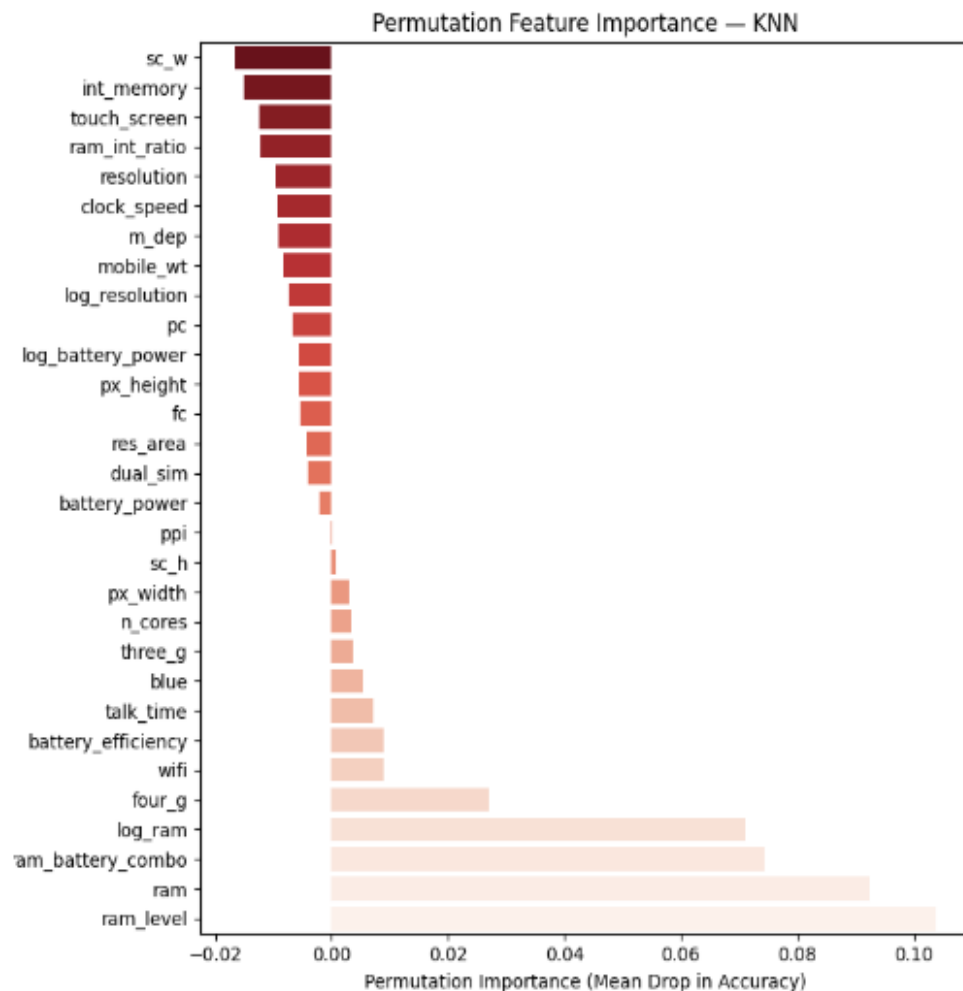
perm = permutation_importance(
    knn_model,
    X_test,
    y_test,
    n_repeats=30,          # repeat count for stability
    random_state=42,
    n_jobs=-1
)

importances = perm.importances_mean
df_knn_imp = pd.DataFrame({
    "feature": X_train.columns,
    "importance": importances
}).sort_values("importance", ascending=False)

print("\nKNN - Full Feature Importance Table:")
print(df_knn_imp.to_string(index=False))

# Plot all features
plt.figure(figsize=(8, max(4, 0.25 * len(df_knn_imp))))
sns.barplot(
    x=df_knn_imp["importance"][::-1],
    y=df_knn_imp["feature"][::-1],
    palette="Reds_r"
)
plt.xlabel("Permutation Importance (Mean Drop in Accuracy)")
plt.title("Permutation Feature Importance - KNN")
plt.tight_layout()
plt.show()
```

:-

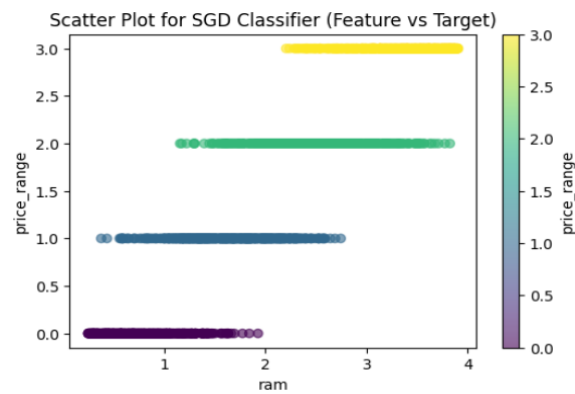


● SCATTER PLOT:-

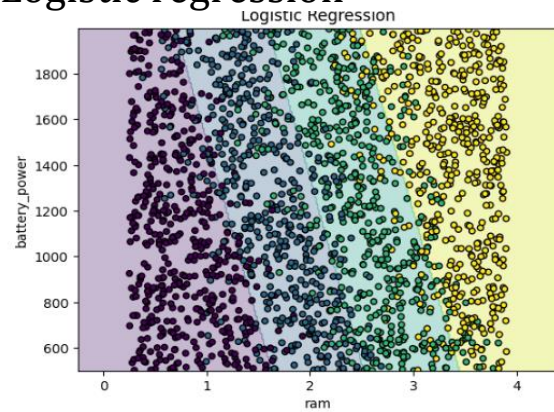
A scatter plot is a graphical representation that shows the relationship between two numerical variables by plotting data points on an X-Y axis.

- Each dot represents one data sample.
- It helps identify **patterns, trends, correlations, and outliers** in the data.
- In this project, scatter plots are useful to visualize how features like **RAM, battery power, or internal memory** relate to the **mobile price range**.

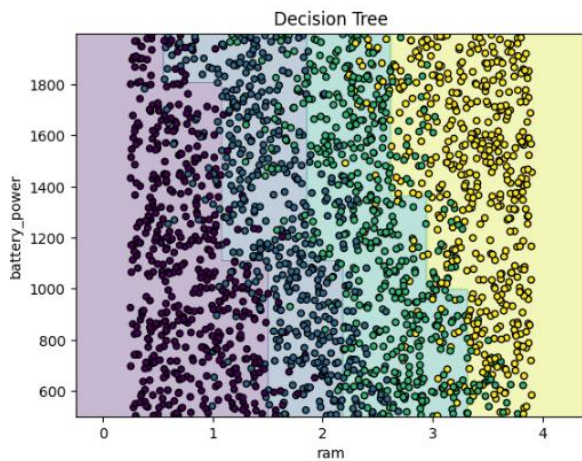
Linear classifier



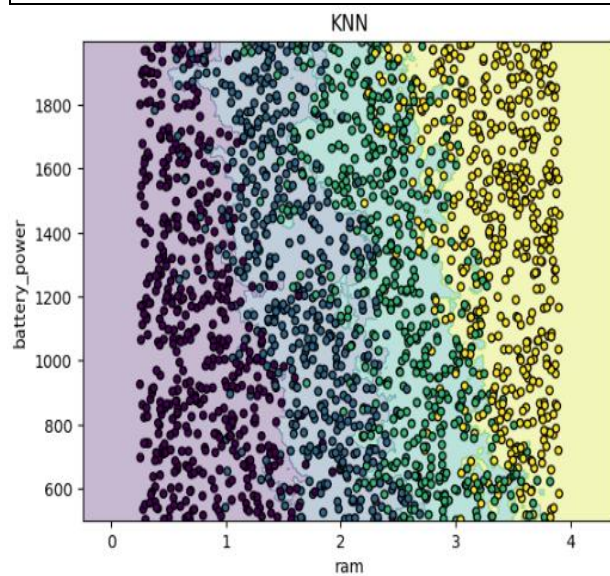
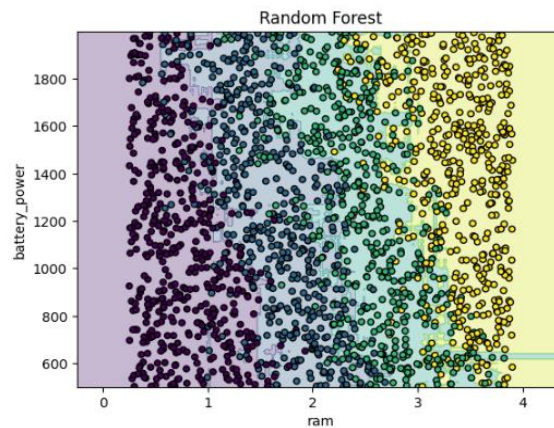
Logistic regression



Decision tree



Random forest



All 5 Model Comparison

● Model-wise Evaluation

1. SGD Classifier -linear

Test Accuracy: 92.75%

Error %: 7.25%

Advantages:-

- Simple and fast to train
- Works well with standardized features
- Interpretable coefficients
- Good baseline model

Disadvantages:-

- Assumes linear decision boundaries
- Slightly lower performance on overlapping classes (1 & 2)
- Less flexible than non-linear models

Observation:-

- Performs well but misclassifies mid-range price classes slightly more.
-

2. Logistic Regression

Test Accuracy: 97.5% (Highest)

Error %: 2.5% (Lowest)

Advantages:-

- Best overall accuracy, precision, recall, and F1-score
- Stable and consistent across all classes
- Handles multi-class classification very well
- Highly interpretable coefficients
- Low overfitting risk

Disadvantages:-

- Assumes linear separability in feature space
- Cannot capture very complex non-linear relations (without kernels)

Observation:-

- Almost perfect confusion matrix

- Balanced performance across **all price ranges**
-

3. Decision Tree

Test Accuracy: 88.5%

Error %: 11.5%

Advantages:-

- Easy to visualize and interpret
- Captures non-linear relationships
- No need for feature scaling

Disadvantages:-

- Prone to overfitting
- Sensitive to small data changes
- Lower generalization accuracy

Observation:-

- Struggles with class 1 and class 2 separation
-

4. Random Forest

Test Accuracy: 93.75%

Error %: 6.25%

Advantages:-

- Handles non-linear patterns well
- Less overfitting than decision tree
- Good feature importance insights
- Robust and stable

Disadvantages:-

- More computationally expensive
- Less interpretable than Logistic / Linear
- Slightly lower accuracy than Logistic Regression

Observation:-

- Very strong performer, but still behind Logistic Regression
-

5. K-Nearest Neighbors (KNN)

Test Accuracy: 82.5%

Error %: 17.5% (**Highest**)

Advantages:-

- Simple concept
- No training phase
- Captures local patterns

Disadvantages:-

- Sensitive to feature scaling and noise
- High prediction time
- Performs poorly in high-dimensional data
- Lowest accuracy among all models

Observation:-

- Confusion between mid-range classes is high

2. Comparison Table (Final Test Results)

Model	Accuracy	Error %	Precision	Recall	F1 Score
Logistic Regression	97.5%	2.5%	0.98	0.97	0.97
Random Forest	93.75%	6.25%	0.94	0.94	0.94
SGDClassifier-linear	92.75%	7.25%	0.93	0.93	0.93
Decision Tree	88.5%	11.5%	0.89	0.89	0.89
KNN	82.5%	17.5%	0.83	0.82	0.83

Comparative Analysis

- **Logistic Regression** achieved the highest accuracy and lowest error rate.
- **Random Forest** performed well but slightly underperformed compared to Logistic Regression.
- **SGDClassifier-linear** showed strong results but lacked flexibility compared to Logistic Regression.
- **Decision Tree** suffered from overfitting and lower generalization.
- **KNN** performed the worst due to high dimensionality and sensitivity to feature scaling.

Across all models, Logistic Regression showed the **most balanced precision, recall, and F1-score**, indicating stable and reliable predictions across all price categories.

Best Model Selection

Selected Model: Logistic Regression

Justification:

Logistic Regression was selected as the best model because:

1. It achieved the **highest test accuracy (97.5%)**
 2. It had the **lowest error percentage (2.5%)**
 3. It showed **consistent performance across all classes**
 4. Precision, recall, and F1-score were uniformly high
 5. The confusion matrix showed minimal misclassification
 6. The model generalized well during cross-validation
 7. It is computationally efficient and easy to interpret
-

Conclusion

In this project, a comprehensive comparison of five machine learning classification models—**SGD Linear Classifier**, **Logistic Regression**, **Decision Tree**, **Random Forest**, and **K-Nearest Neighbors (KNN)**—was performed to predict mobile phone price ranges. Each model was trained, tuned, and evaluated using the same dataset and evaluation strategy to ensure a fair comparison.

Model-Wise Performance Summary

- **SGD Linear Classifier** demonstrated strong baseline performance with good accuracy and interpretability. However, due to its linear nature, it showed minor confusion between mid-range price classes.
 - **Logistic Regression** emerged as the **best-performing model**, achieving the **highest test accuracy (97.5%)** and the **lowest error rate (2.5%)**. It delivered consistently high precision, recall, and F1-score across all price categories, indicating stable and reliable predictions.
 - **Decision Tree** effectively captured non-linear patterns but suffered from overfitting and lower generalization performance, especially when distinguishing between class 1 and class 2.
 - **Random Forest** improved upon Decision Tree performance by reducing overfitting and providing robust predictions. Although it performed strongly, its accuracy remained slightly lower than Logistic Regression.
 - **K-Nearest Neighbors (KNN)** recorded the lowest accuracy due to its sensitivity to feature scaling, noise, and high dimensionality. It struggled significantly with mid-range class separation.
-

Comparative Analysis

The comparative evaluation clearly highlights that:

- **Logistic Regression achieved the highest accuracy and lowest error rate.**
- **Random Forest** performed well but did not surpass Logistic Regression.
- **SGD Linear Classifier** delivered competitive results but lacked non-linear flexibility.

- **Decision Tree** faced overfitting challenges.
- **KNN** underperformed due to data complexity and dimensionality.

Across all evaluation metrics—**accuracy, precision, recall, and F1-score**—Logistic Regression maintained the most balanced and consistent performance.

Best Model Selection

Selected Model: Logistic Regression

Justification:

Logistic Regression was selected as the final model because:

1. It achieved the **highest test accuracy (97.5%)**
 2. It had the **lowest error percentage (2.5%)**
 3. It showed **balanced precision, recall, and F1-score**
 4. It produced a **nearly perfect confusion matrix**
 5. It generalized well during **cross-validation**
 6. It is **computationally efficient**
 7. It offers **high interpretability**, making results easy to explain
-

Final Remark

This project demonstrates that **careful data preprocessing, feature engineering, and model selection** play a crucial role in achieving high predictive performance. Among all tested models, **Logistic Regression proved to be the most reliable and effective approach** for mobile price range classification.

CONCLUSION

In this project, a comprehensive comparison of five machine learning classification models—**SGD Linear Classifier**, **Logistic Regression**, **Decision Tree**, **Random Forest**, and **K-Nearest Neighbors (KNN)**—was performed to predict mobile phone price ranges. Each model was trained, tuned, and evaluated using the same dataset and evaluation strategy to ensure a fair comparison.

Model-Wise Performance Summary

- **SGD Linear Classifier** demonstrated strong baseline performance with good accuracy and interpretability. However, due to its linear nature, it showed minor confusion between mid-range price classes.
 - **Logistic Regression** emerged as the **best-performing model**, achieving the **highest test accuracy (97.5%)** and the **lowest error rate (2.5%)**. It delivered consistently high precision, recall, and F1-score across all price categories, indicating stable and reliable predictions.
 - **Decision Tree** effectively captured non-linear patterns but suffered from overfitting and lower generalization performance, especially when distinguishing between class 1 and class 2.
 - **Random Forest** improved upon Decision Tree performance by reducing overfitting and providing robust predictions. Although it performed strongly, its accuracy remained slightly lower than Logistic Regression.
 - **K-Nearest Neighbors (KNN)** recorded the lowest accuracy due to its sensitivity to feature scaling, noise, and high dimensionality. It struggled significantly with mid-range class separation.
-

Comparative Analysis

The comparative evaluation clearly highlights that:

- **Logistic Regression achieved the highest accuracy and lowest error rate.**
- **Random Forest** performed well but did not surpass Logistic Regression.
- **SGD Linear Classifier** delivered competitive results but lacked non-linear flexibility.
- **Decision Tree** faced overfitting challenges.

- **KNN** underperformed due to data complexity and dimensionality.

Across all evaluation metrics—**accuracy, precision, recall, and F1-score**—Logistic Regression maintained the most balanced and consistent performance.

Best Model Selection

Selected Model: Logistic Regression

Justification:

Logistic Regression was selected as the final model because:

1. It achieved the **highest test accuracy (97.5%)**
 2. It had the **lowest error percentage (2.5%)**
 3. It showed **balanced precision, recall, and F1-score**
 4. It produced a **nearly perfect confusion matrix**
 5. It generalized well during **cross-validation**
 6. It is **computationally efficient**
 7. It offers **high interpretability**, making results easy to explain
-

Final Remark

This project demonstrates that **careful data preprocessing, feature engineering, and model selection** play a crucial role in achieving high predictive performance. Among all tested models, **Logistic Regression proved to be the most reliable and effective approach** for mobile price range classification.
